



UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Course Name: Software Engineering Lab

Course Code: CSE 314

Date of Submission: 22-05-2026

Submitted by:

Rokonur Zaman Rokon – 23101194

System Designer / Software Architect & QA Engineer

Sunandan Das – 23101218

Software Developer (Programmer)

Md. Sakif Hossain – 23101219

Project Manager

Md Minhazul Islam – 23101207

System Analyst / Business Analyst

Submitted to:

Tanjina Helaly

Assistant Professor

Department of CSE, UAP

Final Report

Athletix

(Student Intelligence for Athletic Fitness & Health)



Document Version: 1.0

Date: May 2026

Prepared By: Sunandan Das

Reviewed By: Full Team Approved

TABLE OF CONTENTS

1. Introduction	6 -12
• 1.1 Background of the Project	
• 1.2 Problem Statement	
• 1.3 Objectives	
• 1.4 Scope of the System	
• 1.5 CEP Analysis	
2. Project Management Plan	13-16
• 2.1 Individual Contribution to the Project	
• 2.2 Process Model Used	
• 2.3 Risk Analysis	
3. Requirement Analysis and Specification	17-27
• 3.1 User Requirement	
• 3.2 Functional Requirements (FR)	
• 3.3 Non-Functional Requirements (NFR)	
• 3.4 Hardware & Software Requirements	
4. System Design	27-43
• 4.1 System Architecture Diagram	
• 4.2 Use Case Diagram	
• 4.3 Data Flow Diagram (DFD)	
• 4.4 Database Design (ER Diagram)	
5. Implementation Plan	44-54
• 5.1 Module Breakdown	
• 5.2 Iterative Development Plan	
• 5.3 Weekly Lab Session Work Distribution	
• 5.4 Technology Stack	
• 5.5 Development Methodology	
6. Performance Optimization and Scalability Plan	55-63

- **6.1** Project Snapshot
- **6.2** Current Risks and Potential Bottlenecks
- **6.3** Immediate Improvements
- **6.4** Performance Profiling Process
- **6.5** Short-Term Improvements
- **6.6** Long-Term Scalability Plan
- **6.7** Conclusion

7. Testing and Sustainability Plan 63 - 71

- **7.1** Introduction
- **7.2** Test Objectives
- **7.3** Test Strategy
- **7.4** Test Cases by Module
- **7.5** Test Coverage Summary
- **7.6** Pass / Fail Criteria
- **7.7** Test Schedule
- **7.8** Sustainability Plan
- **7.9** Tools & Dependencies
- **7.10** Test Conclusion

8. Results and Discussion 71 - 94

- **8.1** Final output screenshots
- **8.2** System performance
- **8.3** Club System Development Issue
- **8.4** Testing Differentiation Issue

9. Conclusion 94 - 95

11. Appendices 95 - 98

1. Introduction

In the contemporary sports landscape, the management of athletic organizations has evolved into a highly complex, multi-disciplinary undertaking that requires a delicate, holistic balance between peak physical performance, mental wellness, operational efficiency, and comprehensive medical oversight. Historically, athletic departments, schools, and professional clubs have operated in organizational silos, relying on fragmented, disconnected tools for workout tracking, health records, and administrative compliance—a systemic issue that inevitably leads to communication gaps, data fragmentation, and a significantly higher risk of athlete burnout or mismanaged injury rehabilitation. To address these widespread inefficiencies and meet the demands of modern sports ecosystems, **Athletix** serves as an enterprise-grade, role-based sports management and athlete development platform designed to unify the full lifecycle of student-athlete care into a single, comprehensive workflow. By establishing a centralized digital infrastructure with distinct, secure operational roles, the platform effectively bridges the gap between performance optimization and institutional governance: it empowers athletes with self-service tools to manage their routines and profiles, enables coaches to design targeted, data-driven training regimens, allows medical staff to accurately monitor health metrics and oversee rehabilitation, and provides administrators with robust control over user compliance and credentials. Ultimately, Athletix replaces legacy, multi-tool setups with a streamlined, data-driven environment, enabling sports organizations to eliminate departmental silos, optimize operational workflows, and cultivate elite athletic excellence while strictly safeguarding holistic student-athlete well-being.

1.1 Background of the project

Athletix is a web-based student-athlete management system developed using Django to centralize athletic training, performance tracking, and health monitoring within a single platform. The system is designed for schools, sports academies, and athletic organizations that require an efficient way to manage athletes, coaches, medical staff, and administrative activities. Instead of using separate tools for communication, training schedules, and health records, Athletix integrates these functions into one organized application. The platform follows a role-based access system where athletes, coaches, medical staff, and administrators have different responsibilities and permissions. Athletes can manage profiles, access routines, track progress, and review health feedback. Coaches can approve requests, assign training routines, and monitor athlete performance. Medical staff can maintain health records, monitor injuries and recovery, and provide medical feedback, while administrators oversee user management and system operations.

Athletix supports the complete athlete development process by combining training management with wellbeing monitoring. The platform includes features for performance tracking, health observation, recovery management, and internal communication between

users. By integrating these activities into one system, Athletix improves coordination, organization, and overall efficiency in sports program management.

1.2 Problem statement

Athletix is designed to centralize athlete development, coaching workflows, and medical supervision within a single web-based platform. At present, sports organizations often depend on fragmented tools such as spreadsheets, messaging applications, and paper-based records to manage training schedules, coaching assignments, health monitoring, and injury recovery. This lack of integration leads to data silos, delayed communication, inconsistent record-keeping, and limited visibility across stakeholders.

Consequently, coaches do not have a unified view of athlete performance and health status, medical staff struggle to correlate injuries with training loads, athletes must navigate multiple channels for routines and feedback, and administrators face difficulties in ensuring proper approvals and data consistency. These challenges reduce training efficiency, increase injury risk, and hinder long-term performance analysis.

Athletix addresses these issues by providing a centralized, role-based system for athletes, coaches, medical staff, and administrators. It integrates training management, health tracking, structured feedback, and real-time communication into a single platform. The system aims to eliminate data fragmentation, improve coordination, and support faster, more informed decision-making to enhance athlete safety and development.

1.3 Objectives:

The primary objective of **Athletix** is to centralize and standardize athlete management within a single, unified digital ecosystem. Designed for academic institutions, sports clubs, and athletic organizations, the platform streamlines the intersection of performance optimization, training coordination, healthcare monitoring, and stakeholder communication. By replacing fragmented, multi-tool setups with a structured workflow, Athletix enhances operational coordination and provides comprehensive data visibility across the entire lifecycle of student-athlete care.

- **Role-Based User:** Establish a secure, multi-tenant framework to manage distinct profiles and access controls for athletes, coaches, medical personnel, and system administrators.
- **Athletic & Performance Tracking:** Provide robust tools to monitor sports participation, log development milestones, and evaluate historical performance data.
- **Coaching & Routine Management:** Empower coaching staff to design daily training regimens, manage roster relationships, and efficiently process athlete training requests.

- **Medical Supervision & Rehabilitation:** Enable healthcare staff to track vital health metrics, document injuries, monitor recovery trajectories, and input clinical feedback relative to an athlete's physical readiness.
- **Centralized Communication Infrastructure:** Facilitate secure, internal messaging protocols to ensure fluid, real-time collaboration among all platform stakeholders.
- **Administrative Oversight & Compliance:** Arm administrators with overarching system controls, including account vetting, user management, and platform-wide auditing tools.

Ultimately, the Athletix project bridges the gap between athletic development and institutional governance, providing a data-driven environment that maximizes competitive excellence while strictly safeguarding athlete welfare.

1.4 Scope of the system:

The scope of work defines the functional boundaries, core modules, and explicit exclusions for the development of the Athletix sports management and athlete development platform.

1.4.1. Functional Scope

Module 1: Identity & Access Management

- **Authentication Framework:** Secure user registration, multi-tenant login pathways, password reset functionality, and account management.
- **Role-Based Access Control (RBAC):** Implementation of rigid permission tiers for four distinct user classes:
 - **Athletes:** Profiles containing sport type, fitness metrics, biometric data (age, height, weight), and club affiliations.
 - **Coaches:** Profiles documenting specialization, years of experience, certifications, and assigned sports.
 - **Medical Staff:** Profiles capturing professional licensing data and medical specialties.
 - **Administrators:** Global access control and system-wide configurations.
- **Vetting Workflow:** Restricted account activation for Coach and Medical roles, requiring manual verification and approval by an Administrator.

Module 2: Sports & Organization Architecture

- **Sports Directory:** A global database for managing sport types, categorizations, and descriptions.

- **Club Management:** Tools allowing coaches to provision, configure, and manage distinct athletic clubs.
- **Roster Integration:** Systems enabling athletes to select sports disciplines and enroll in active clubs.

Module 3: Training & Mentorship Relationships

- **Relationship Brokerage:** A request-and-approval mechanism allowing athletes to solicit coaching relationships (Pending, Accepted, Rejected states).
- **Data Mapping:** One-to-many database relationships linking a single coach to multiple athletes.
- **Lifecycle Tracking:** Historic logging of training associations, capturing start/end dates and real-time active statuses.

Module 4: Training & Routine Optimization

- **Routine Engine:** Authoring tools for coaches to build day- or date-bound daily routines detailing titles, exercise blocks, execution times, and programmatic notes.
- **Execution Workflow:** Self-reporting interface for athletes to mark routines as complete, paired with a supervisor review and approval workflow for coaches.
- **Performance Logs:** Database tracking of historical routine completion and basic performance metrics.

Module 5: Health Monitoring

- **Clinical Telemetry:** Secure interfaces for medical staff to record vital signs and wellness metrics (heart rate, blood pressure, weight, sleep metrics, and fatigue levels).
- **Injury & Recovery Triage:** Standardized tracking matrices to log injury severity (*None, Minor, Moderate, Severe, Recovering*) and recovery status (*Good, Needs Monitoring, Critical*).
- **Patient Self-Reporting:** Features allowing athletes to log daily biometric data, coupled with a formal "Review Request" trigger to alert medical staff of anomalies.

Module 6: Medical Feedback Loop

- **Feedback Categorization:** Structured clinical notation separating performance-related advice from critical health recommendations.
- **Audit-Ready Logs:** Timestamps and cryptographically linked medical entries mapping specific health feedback directly to an athlete's permanent medical record.

Module 7: Unified Messaging System

- **Peer-to-Peer Chat:** Secure, asynchronous, one-to-one internal text communication.
- **Message Telemetry:** Read/unread status indicators and persistent conversation histories.
- **Gateway Permissions:** Role-enforced communication guardrails (e.g., controlling which tiers can initiate contact with medical personnel).

Module 8: User Experience & Front-End Interfaces

- **Marketing & Auth Gateways:** Landing page, informational views, and secure login/registration screens.
- **Tailored User Dashboards:** Responsive, role-specific home screens optimized for the daily tasks of Athletes, Coaches, Medical Staff, and Admins.
- **Unified Interface Ecosystem:** Profile views, data configuration tables, and an integrated chat UI.

1.5 CEP Analysis:

Complex Engineering Problem Attributes (P) :

Code	Attribute Name	Description
P1	Depth of Knowledge Required	Fits because the project depends on deep software engineering knowledge to build a working system with models, views, templates, authentication, and role-based workflows.
P2	Range of Conflicting Requirements	The system has conflicting requirements between athletes, coaches, medical staff, and administrators.that matches with this part.
P3	Depth of Analysis Required	Fits because the system has no single obvious solution and requires custom workflow design across multiple modules.
P6	Involvement of Diverse Stakeholders	Fits because the project involves athletes, coaches, medical staff, and administrators with different permissions and needs.

P7	Interdependence of Components	Fits because the system contains many interconnected modules such as user management, chat, routines, and health tracking.
----	-------------------------------	--

Complex Engineering Activities (A) :

Code	Activity Name	Description
A1	Range of Resources	This requirement is met because the project uses diverse resources including people, databases, templates, media, external libraries, and development tools.
A4	Consequences to Society and Environment	This applies because the system manages athlete health and recovery data, which may affect safety and decision-making.
A5	Familiarity of Issues	Fits because the project goes beyond simple previous projects and applies principles-based workflows for training, medical review, and communication.

Knowledge Profile (K):

Code	Knowledge Profile Element	Description
K2	Mathematics, Numerical Analysis, Statistics, and Computing Fundamentals	This criterion is satisfied because the project uses software and data concepts such as database models, forms, validation, and logic processing.

K4	Specialist Knowledge for Engineering Practice	This is demonstrated because the project applies specialist knowledge of Django web development and system architecture.
K5	Engineering Design Knowledge	The project aligns here because the project focuses on designing complete systems and workflows rather than only individual pages.
K6	Engineering Practice and Tools	This standard is met because the project uses tools and technologies such as Django, SQLite, Selenium, Chart.js, and Pillow.
K7	Comprehension of Role of Engineering in Society	This comes into play because the system includes privacy, health data protection, responsibility, and role-based access considerations.
K8	Engagement with Research Literature and Knowledge Development	This is applicable weakly to moderately because the project includes testing and performance-related work, though it is not heavily research-oriented.

2 Project Management Plan

2.1 Individual Contribution to the Project

Individual contributions to the project are shown in Table 2.1.

Member Name	Requirement Specification	Planning	Designing	Model Building	User Interface	Testing	Deployment
Sunandan Das	✓	✓	✓	✓	✓	✓	✓
Rokonur Zaman Rokon			✓	✓		✓	
Md. Sakif Hossain	✓	✓					

2.2 Process Model Used

In this project, the **Prototyping Process Model** was used for developing the ATHLETIX System. The prototyping model helped the development team gradually build, test, and improve the system based on continuous feedback and requirement analysis. This approach was suitable for the project because the system required multiple user roles, dashboards, performance monitoring features, and interactive functionalities.

The prototyping paradigm consists of the following stages:

- **Communication**

In this stage, discussions were conducted with team members to identify the main objectives and requirements of the ATHLETIX System. The required functionalities for athletes, coaches, medical staff, and administrators were analyzed. System requirements such as authentication, performance tracking, medical monitoring, chat system, and club management were gathered and documented.

- **Quick Plan**

After gathering requirements, a quick planning phase was completed to determine:

- Project modules
- Sprint schedules

- Technology stack
- Database structure
- Task distribution among team members

The development timeline and weekly sprint activities were also planned during this phase.

• **Modeling Quick Design**

In this stage, preliminary system designs and models were prepared. Different diagrams and prototypes were created including:

- Use Case Diagram
- Data Flow Diagram (DFD)
- ER Diagram
- System Architecture
- Figma UI Prototype

The quick design helped visualize the overall workflow and user interactions of the ATHLETIX System.

• **Construction of Prototype**

During this phase, the initial prototype of the ATHLETIX System was developed using:

- Python Django
- HTML, CSS, JavaScript
- Chart.js
- SQLite Database

Core functionalities such as login, dashboards, training management, medical monitoring, chat system, alerts, and club management were implemented incrementally.

• **Deployment, Delivery and Feedback**

After implementation, the prototype was tested and reviewed regularly. Feedback was collected from team members and improvements were made continuously. Unit Testing and Selenium Testing were performed to validate system functionality and usability.

Based on the feedback, several modules such as:

- Performance charts
- Club management
- Admin system
- Notification system

were updated and improved before final deployment.

• **Communication**

Continuous communication was maintained throughout the project development process. Team discussions, debugging sessions, testing feedback, and progress evaluations helped ensure successful completion of the ATHLETIX System.

2.3 Risk Analysis

The ATHLETIX System is a web-based sports management and athlete monitoring platform developed using Django. During the development and deployment process, several potential risks were identified related to technical implementation, security, performance, testing, and project management. Proper mitigation strategies were considered to reduce the impact of these risks.

Risk Category	Risk Description	Impact	Mitigation Strategy
Requirement Risk	Changes in user requirements during development may affect project scope and implementation.	High	Continuous communication and iterative development approach were followed.
Technical Risk	Difficulty in integrating performance charts, dashboards, and analytics modules.	Medium	Used Chart.js documentation, GitHub resources, and incremental testing.
Database Risk	SQLite may face concurrency and scalability limitations in large-scale deployment.	High	Planned migration to PostgreSQL for production deployment.

Security Risk	Unauthorized access or role mismanagement may expose sensitive user data.	High	Implemented role-based access control and secure authentication system.
Testing Risk	Confusion between Unit Testing and Selenium Testing could affect test management.	Medium	Used separate tagging and categorized testing structure for independent execution.
Performance Risk	Large database queries and dashboard analytics may slow down the system.	Medium	Used Django ORM optimization techniques such as <code>select_related</code> and <code>prefetch_related</code> .
Deployment Risk	Incorrect server or environment configuration may affect deployment stability.	Medium	Maintained environment configuration through Django settings and deployment planning.
Data Loss Risk	Unexpected database or system failure may cause data loss.	High	Planned backup and recovery strategy for database protection.
Team Coordination Risk	Miscommunication among team members may delay development progress.	Low	Weekly sprint planning and continuous team communication were maintained.
UI/UX Risk	Poor user interface responsiveness may affect user experience.	Medium	Used Figma prototyping and frontend testing for UI improvements.
Maintenance Risk	Future feature updates may become difficult without proper modular structure.	Medium	Used modular Django application architecture for maintainability and scalability.

3. Requirement Analysis and Specification:

3.1 User Requirement

- **Overview:** The Athletix platform functions as a centralized, secure, multi-tenant digital ecosystem providing role-based workflows specifically architecturalized for sports organizations. The application establishes strict communication and data boundaries that allow distinct stakeholders—specifically administrators, athletic coaches, individual players, and certified medical professionals—to manage physical preparation tracks, log and interpret physiological performance metrics, compile authenticated clinical health records, and carry out audited internal communications.
- **Primary Actors:**
 - **Admin (System Administrator):** Operates with global superuser credentials to oversee the systemic lifecycle of all user accounts. This actor regulates application configurations, manages data models, holds exclusive rights over account state transitions (e.g., executing manual profile approvals or permanent database drops), monitors platform telemetry, and reviews comprehensive activity audit logs.
 - **Coach:** Acts as the primary orchestrator of physical training regimens. This actor maintains rights to build modular exercise structures, schedule multi-week training pipelines, bind templates to specific athletes or full team groups, evaluate historical physical progress, and read qualitative performance logs via automated analytics dashboards.
 - **Player (Athlete):** Represents the core end-user and recipient of training assignments. This actor accesses a personalized schedule portal to view outstanding physical tasks, inputs quantitative workout completion metrics (e.g., sets completed, repetitions executed, weights lifted), logs daily subjective fatigue indicators, and references historical performance charts.
 - **Medical Staff:** Consists of certified physical therapists and medical practitioners. This actor operates inside an isolated system boundary to initialize individual patient health charts, append diagnostic data files, request formal physical reviews, and change overarching athletic fitness clearance statuses.
 - **Support/Chat (Contextual Messaging Engine):** Works as an internal infrastructure micro-utility that facilitates threaded, text-based dialogues and multi-media document sharing. This engine enables direct, authenticated communication

channels specifically bound to system entities like explicit routines or medical records.

- **User Goals:**

- **Admin:** Enforces high application uptime standards, ensures zero-leakage security boundaries across distinct roles, audits data transactions, and handles structural account creation requests.
- **Coach:** Builds targeted, data-backed training templates, minimizes physical preparation drop-offs, catches athletic performance regressions, and provides immediate contextual training feedback.
- **Player:** Tracks daily training objectives, logs performance and physical data without interface friction, views personal health trendlines, and maintains secure interaction channels with administrative and physical guidance staff.
- **Medical Staff:** Protects clinical patient privacy rights, maintains accurate treatment and injury records, controls athletic participation flags based on physical readiness, and secures confidential health observations from coaching and player accounts.

- **Acceptance Criteria:**

- Role-Based Access Control (RBAC) layers intercept unauthorized incoming requests at the routing layer, view controller layer, and raw database query layer, immediately aborting execution and serving standard HTTP 403 Forbidden screens.
- Binary data objects (including profile imagery, diagnostic documents, and medical attachments) are processed through file header verification checks, cleared of malicious metadata structures, stored in isolated folders, and made retrievable solely by authenticated accounts with matching permission records.

3.2 Functional Requirements (FR)

- **FR-1: User Account Registration & Role Selection** The identity intake system exposes a registration web interface for unauthenticated platform visitors. The module securely collects a user's verified institutional email address, full alphabetical legal name, a cryptographically complex user password, and an explicit role selection

indicator defining the account type as Coach, Player (Athlete), or Medical Staff. The application executes real-time database lookups to guarantee email uniqueness across the tables and automatically routes newly created profiles into an unverified, zero-access holding state pending downstream administrative validation.

- **FR-2: Cryptographic Authentication & Multi-Factor Login** The core login engine evaluates user credentials by processing incoming password strings against stored database states utilizing industry-standard cryptographic hashing algorithms (Argon2 or PBKDF2). Upon passing baseline verification checks, the application coordinates multi-factor challenges using time-based one-time password (TOTP) protocols. A self-service account restoration module tracks account states and generates time-restricted, single-use email cryptographic tokens to enable secure, unauthenticated password resets.
- **FR-3: Active Middleware Role-Based Access Control (RBAC) Enforcement** The platform employs strict access control middleware and declarative routing decorators that evaluate a user's session group token before resolving any user interface view layout or processing any API payload. Attempted access to an Admin-only configuration utility or a protected Medical sub-module folder by an account missing matching credentials (such as a Player or Coach) halts query execution, writes a high-priority entry to the infrastructure security log, and drops the client connection with an explicit HTTP 403 Forbidden payload.
- **FR-4: Administrative Account Approval & Lifecycle Control Panel** The administration application provides a centralized operational queue that indexes all unverified and newly modified user registration records. The system actively restricts login capabilities and blocks all backend data queries for these accounts until an active administrator manually updates their status indicator to 'Approved'. Administrators modify user account states between 'Active', 'Suspended', or 'Deleted' to instantly drop active sessions tied to those identification keys.
- **FR-5: Profile Personalization & Multi-Media Header Processing** Authenticated users utilize a personal settings dashboard to update primary contact parameters, regional preferences, and training background details (such as age, primary sport, playing position, and initial training level classification). This module leverages a dedicated server-side image processing utility (Pillow) to evaluate uploaded profile photos, validate binary file signatures, strip hidden exchangeable image file format (EXIF) metadata strings, resize canvases to uniform avatar dimensions, and write the final file down to secure, restricted media paths.
- **FR-6: Training Routine and Exercise Template Creation Engine** The coaching module exposes an interactive form canvas used to construct multi-tiered training routines

and physical regimens. A routine is saved as a discrete collection of structured exercise items, where each step captures the targeted exercise name, textual instructional descriptions, targeted set thresholds, precise repetition boundaries, required weight parameters (supporting both metric and imperial units), and mandatory rest interval timers. These configurations save as reusable master templates within the relational database.

- **FR-7: Routine Assignment, Scheduling, & Immutability Framework** The routine engine links training templates to individual player accounts or collective team rosters across explicit calendar date ranges, performance targets, and completion deadlines. When an assignment is deployed, the system saves a standalone snapshot copy of the routine parameters to an assignment table, statefully locking its core data structure to ensure that subsequent coach modifications to the master template do not retroactively alter or corrupt active training records.
- **FR-8: Real-Time Athlete Workout Logging & Interaction Interface** Players track active, past-due, and finished training tasks through a unified chronological calendar dashboard. Activating an assignment loads a dedicated data logging interface where the athlete checks off completed sets, records actual weights lifted, updates repetition achievements, inputs sport-specific performance dimensions (including drills, movement tracking, shot accuracy, and ball spin metrics for table tennis), logs session timestamps, and appends qualitative remarks regarding localized physical execution difficulties.
- **FR-9: Daily Subjective Wellness and Recovery Metrics Intake** The system runs an isolated evaluation routine that prompts active players to submit a comprehensive daily subjective health assessment once every 24 hours. The intake portal records scale-based integers mapping sleep duration quantities, sleep quality indices, muscle soreness severity ratings (using a standardized 1-10 scale), systemic energy reserves, psychological stress and mental fatigue parameters, and resting heart rates independently of specific workout logs.
- **FR-10: Analytical Progress Visualization Dashboards** The platform renders interactive graphical timeline charts on both coach and player dashboards using Chart.js over frontend layers. The visualization subsystem queries historical data arrays directly from the database, aggregating information to chart total training volume curves, estimated one-rep max (1RM) progressions, routine scheduling compliance ratios, and rolling averages of subjective wellness metrics over adjustable 7-day, 30-day, or custom date ranges.
- **FR-11: Confidential Medical Record Initialization & Structural Logs:** A separated sub-module completely isolates sensitive health information under the exclusive access rights of the Medical Staff role. This interface allows an authorized medical

practitioner to initialize a permanent clinical folder for an athlete to document historical injury charts, active physical therapy trackers, therapeutic use exemptions (TUEs), heart condition records, and specialized consultation annotations hidden from coaches and administrators.

- **FR-12: State-Driven Athletic Clearance Status Management Workflow:** The medical app provides a control dashboard to modify an athlete's global physical participation clearance flag within a hardcoded state machine: 'Fully Cleared', 'Restricted/Modified Duty', or 'Completely Sidelined'. Shifting an athlete to a restricted or sidelined state automatically fires a notification indicator to the assigned coach's interface and locks the player's account from opening heavy physical training routines or intensive training sessions.
- **FR-13: Protected Medical Document Upload & Secure Link Delivery:** The medical module includes a secure binary file attachment handler to bind diagnostic imaging scans, laboratory results, and external specialist PDF charts to an athlete's health file. The backend checks incoming file types, enforces file size boundaries below 10MB, saves data blobs into non-public application storage directories, and delivers assets to authenticated users via signed, temporary URLs to maintain data privacy.
- **FR-14: Context-Anchored Threaded Messaging Hub:** The platform features an integrated communications utility structured around distinct, threaded discussion entities. Users initialize secure chat records that are explicitly anchored to a system context—such as a targeted training routine template, a specific workout performance log, or a medical review entry. The system saves textual message data strings and displays conversation logs in an audited chronological stream.
- **FR-15: Asynchronous Event-Driven Notification Manager:** An internal background worker tracks core application state transitions to trigger contextual notifications. The notification manager distributes real-time in-app dashboard banners and automated emails when a coach deploys a training routine, a player logs a workout completion, a medical staff member updates an athlete's participation clearance, or a user posts a message to an active conversation thread.
- **FR-16: Tabular Analytical Data Export Facility** Authorized roles (Coaches and Medical Staff within their respective data boundaries) utilize an export panel to extract system information. The utility queries target database tables, maps active filters, formats rows into structured comma-separated values (CSV) streams, and downloads files containing training completions, historical wellness parameters, or administrative telemetry.
- **FR-17: Comprehensive Infrastructure & Security Audit Interceptor** An automated, low-level database interceptor monitors and logs all critical mutations across the application environment. Every password modification, account permission state

change, user profiling adjustment, and medical record query writes an unalterable log line capturing a high-resolution timestamp, actor identification keys, client source IP blocks, target database IDs, and the exact data mutation performed.

- **FR-18: Administrative System Backup & Maintenance Panel** The administrative console contains tools to configure background application maintenance jobs. Administrators trigger manual or scheduled database dumps (targeting local SQLite development files or enterprise relational production database mirrors) and create compressed, password-protected zip archives of application media storage paths.
- **FR-19: Organizational Team & Roster Grouping Matrix** Admin and coach interfaces utilize a logical grouping framework to organize individual player profiles into distinct 'Teams' or 'Rosters'. This data matrix enables multi-user filtering, routes unified training assignments to whole cohorts simultaneously, and broadcasts group notifications without requiring coaches to process adjustments individually for every athlete.
- **FR-20: Multi-Variable Search, Query, & Filtering Engine** The search subsystem indexes data layers across all primary entity management dashboards. The platform parses textual strings to return matching lookups across profiles, routines, medical files, or historic workout logs. The system supports multi-variable filters, enabling users to narrow down data sets by explicit date ranges, assignment completion flags, or player clearance codes.

3.3 Non-Functional Requirements (NFR)

- **NFR-1: Security**

The application enforces transport-layer encryption using HTTPS protocols across all production environments. High-sensitivity database items, including personal identifiers and clinical medical logs, are encrypted at rest using AES-256 standard protocols. API interactions operate strictly via expiring verification tokens, and strict permission decorators protect backend database tables from unauthorized roles.

- **NFR-2: Availability**

The production infrastructure deployment pattern maintains an operational availability uptime target of 99.5%, excluding predefined maintenance windows. The system architecture runs across decoupled application workers behind an active

network load balancer, ensuring that single-node server drops do not cause complete application failure or terminate active client sessions.

- **NFR-3: Performance**

The application layer and optimized query lookups limit standard API response times and page delivery layout generation to less than 2.0 seconds during nominal, non-peak traffic loads. Static resources are compressed through server asset pipelines and repeated database queries are cached to minimize browser parsing overhead and lower data traffic costs.

- **NFR-4: Maintainability**

The platform source code follows a decoupled, modular design pattern, splitting distinct operational units into isolated Django applications including user, player, medical_staff, coach, and chat. Code reliability and stability are validated by automated integration and unit test suites running via Selenium and the native Django testing framework before any updates are pushed to staging or production codebases.

- **NFR-5: Usability**

The user interface layouts adapt dynamically across varying screen resolutions using fluid grids, providing functional accessibility across mobile devices, tablets, and wide-screen desktop displays. The presentation layer utilizes semantic HTML blocks, respects keyboard-driven tab navigation paths, and mandates explicit alternative text tags on media components to comply with global web accessibility standards.

- **NFR-6: Compliance**

The database engine processes data retention and archival rules in alignment with standard healthcare data privacy regulations. Deletion commands targeting personal profiles or health logs execute through a multi-tiered soft-delete pattern, retaining the affected rows inside a highly restricted, unindexed state for a recovery window prior to permanent removal.

- **NFR-7: Reliability**

The platform schedules daily automated backup tasks, saving state updates of the active database state (db.sqlite3 during local operations or relational production clusters) and compressing all media files every 24 hours. Backups upload to isolated,

off-site storage targets alongside verified structural replication and restore guides for disaster recovery.

- **NFR-8: Scalability**

The database schema layer is constructed with abstract Object-Relational Mapping (ORM) classes. This data structure maintains absolute portability across relational database management systems, allowing zero-refactor migrations from the lightweight local SQLite development backend over to high-capacity production database clusters like PostgreSQL or MySQL.

- **NFR-9: Compatibility**

Client scripts, styling classes, and markup templates deliver consistent visual rendering and functional stability across current variations of dominant web platform engines. This requires verification against standard versions of Chromium-driven web browsers (Google Chrome, Microsoft Edge, Brave), WebKit runtime systems (Apple Safari), and Gecko engine targets (Mozilla Firefox).

- **NFR-10: Extensibility**

Service boundaries are built on abstract interfaces and clear internal API entry points. This architectural structure guarantees that additional features, external hardware data integrations, wearable sensors, and third-party communication webhooks insert directly into the codebase without affecting existing apps or triggering unintended side effects.

3.4 Hardware & Software Requirements

3.4.1 Development Environment Requirements (Recommended)

- **Operating System:** Microsoft Windows 10 / Windows 11 (Primary development operating system workstation environment).
- **Processor (CPU):** Intel Core i5 / AMD Ryzen 5 or equivalent (Minimum Dual-Core 2.0 GHz clock speed; Quad-Core processor highly recommended).
- **System Memory (RAM):** 8 GB physical RAM minimum (16 GB recommended to smoothly handle local database pooling, backend compilation testing, and active asset processes).
- **Storage Space (Disk):** Minimum 20 GB of free solid-state drive (SSD) storage capacity for core workspace provisioning.

- **Core Application Software Stack:**
 - Python 3.10 / 3.11 Runtime Execution Environment.
 - Django Web Framework (explicitly pinned via strict project requirements.txt).
 - Pip Package Installer alongside Venv virtual environment isolation utilities.
 - Git Version Control utility for repository tracking and codebase deployment.
 - **Pillow (Python Imaging Library):** Embedded server-side utility used in the backend profile personalization module to parse, validate binary file signatures, strip hidden exchangeable image file format (EXIF) metadata strings, crop canvas paths, and resize image headers.
 - SQLite Database Management System (utilized strictly for rapid local engineering prototypes and internal developer environment data storage).
 - Client-Side Execution Layer: Core HTML5 engine, CSS3 layout stylesheets, and native modern JavaScript processing.

3.4.2 Testing & Staging Environment Requirements

- **Operating System:** Microsoft Windows Server or stable Linux instance (Ubuntu Server 20.04+ LTS environment).
- **Processor (CPU):** Minimum 2-Core Virtual Processor (vCPU) configuration allocation.
- **System Memory (RAM):** 4 GB to 8 GB allocated framework execution memory.
- **Storage Space (Disk):** 50 GB available high-speed structural network storage space.
- **Testing & Staging Software Infrastructure:**
 - Python 3.10 / 3.11 execution runtimes matching the development vector.
 - Integrated Automated Test Suites (Native Django Testing Framework / pytest infrastructure).
 - **Selenium WebDriver Driver Pack:** Cross-browser automated user interface (UI) verification driver components integrated into the Quality Assurance pipeline to automatically execute functional view flows and integration validation tests before deployment.
 - Containerized MySQL Server Engine instance configured to mirror the live database schema.

3.4.3 Production Environment Guidelines

- **Application Node Architecture (Institutional Scale):**

- Hardware Layer: 2 vCPU cores, 4 GB to 8 GB dedicated System RAM, 100 GB High-Speed Solid-State Storage.
- Server Infrastructure: WSGI-compliant application handler (Gunicorn or uWSGI) running behind an active upstream network reverse proxy container (Nginx configuration layer).
- Transport Layer Encryption: Configured Transport Layer Security (TLS/HTTPS standard certificates via automated Let's Encrypt validation pipelines).
- **Database Management System:** Production-grade relational database deployment utilizing a secure, standalone **MySQL Server** cluster to isolate, index, and process transactional and physical application data tables.

3.4.4 Storage & Data Backup Specifications

- **Database Backups:** Automated daily structural data replication routines targeting the target MySQL relational schemas.
- **Media Asset Backups:** Secure local multi-directory replication or institutional file cluster paths designed to archive diagnostic charts, profile avatars, and document file items.
- **Disaster Recovery:** Formally structured, documented, and validated restoration routines to quickly reset state vectors in the event of hardware or system infrastructure failure.

3.4.5 Client Browser & Platform System Compatibility

- **Web Browsers:** Full, non-degraded responsive rendering confirmed on modern browser clients, explicitly optimized for standard versions of **Google Chrome, Mozilla Firefox, and Microsoft Edge**.
- **Interface Prerequisites:** Client browsers must have standard client-side scripting execution enabled to parse interactive data elements, metrics logging systems, and real-time visualization dashboards.

3.4.6 Security Minimums

- **Credential Protection:** Hardened system authentication workflows enforcing strict validation rules for password strings alongside security logs tracking unauthorized request vectors.
- **Access Restraints:** Middlewares and authorization barriers enforcing multi-tiered **Role-Based Access Control (RBAC)** across view contexts, ensuring user data privacy by blocking cross-role path exploitation.

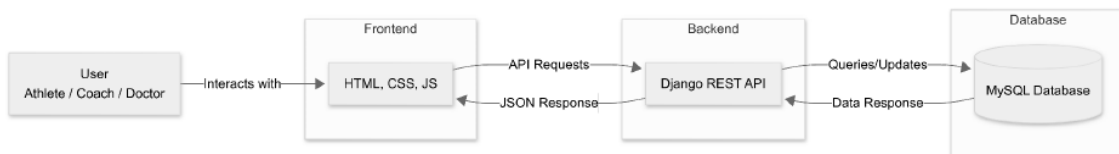
- **Transport Isolation:** Mandatory application of end-to-end transport-layer encryption (HTTPS) isolating all client-server communications.

3.4.7 Targeted Operational Sizing Metrics

- **Small Institutional Bracket (Up to 100 Active Users):** Single core virtual instance (2 vCPU / 4 GB RAM configuration hosting both the active Django instance and a standalone local database system).
- **Medium Bracket (100 to 1,000 Active Concurrent Users):** Distributed dual-core processor infrastructure decoupling the frontend web workspace (2 vCPU / 4 GB RAM application node) from a separate data cluster instance (2 vCPU / 4 GB RAM dedicated MySQL server node).

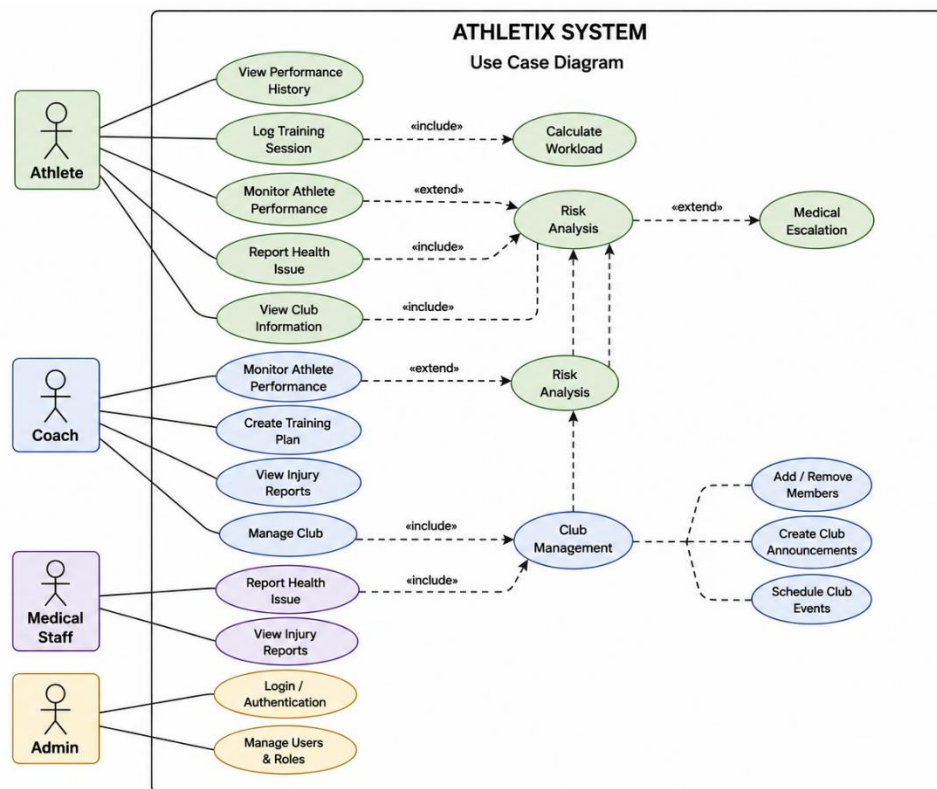
4. System Design

4.1 System Architecture Diagram



The architectural diagrams represent a layered MVC-based design for the Athletix system. In the standard MVC architecture, the Controller handles application logic, the Model manages data and business rules, and the View renders the user interface, all connected to a central database. Building on this concept, the Athletix system follows a Layered MVC Architecture consisting of three main layers. In the Presentation Layer, users such as Athletes, Coaches, and Doctors interact with a frontend developed using HTML, CSS, and JavaScript. The Application Layer processes API requests through a Django REST API backend, where business logic is executed and JSON responses are generated. Finally, the Data Layer manages data persistence, where the backend communicates with a MySQL database to perform queries, retrieve information, and store updates efficiently.

4.2 Use Case Diagram



The Use Case Diagram represents how different users interact with the ATHLETIX System. It shows the main functionalities of the system and the relationships between actors and use cases.

The system contains four main actors:

- Athlete
- Coach
- Medical Staff
- Admin

Each actor performs different activities within the system.

1. Athlete

The Athlete interacts with the system to manage training and monitor personal performance.

View Performance History

Allows athletes to:

- View previous performance records

- Track improvement and progress

Log Training Session

Athletes can:

- Record workout sessions
- Submit training details

This use case includes Calculate Workload, where the system automatically calculates training workload.

Monitor Athlete Performance

Allows athletes to:

- Monitor fitness progress
- Analyze training performance

This process may extend to Risk Analysis if abnormal conditions are detected.

Report Health Issue

Athletes can:

- Report injuries or health problems
- Request medical attention

This use case includes Risk Analysis to evaluate the severity of the issue.

View Club Information

Athletes can:

- View club announcements
- Check club events
- Access club details

2. Coach

The Coach manages athlete training and club activities.

Monitor Athlete Performance

Coaches can:

- Observe athlete progress
- Detect performance risks
- Analyze athlete condition

This may trigger Risk Analysis when needed.

Create Training Plan

Allows coaches to:

- Create workout schedules
- Assign training plans to athletes

View Injury Reports

Coaches can:

- View athlete injury details
- Monitor recovery progress

Manage Club

Coaches can:

- Manage club activities
- Organize members
- Control announcements

This use case includes Club Management.

3. Club Management

The Club Management module handles club-related operations.

It includes:

- Add / Remove Members
- Create Club Announcements
- Schedule Club Events

This feature helps coaches organize athletes and maintain club communication.

4. Medical Staff

Medical Staff manage athlete health information.

Report Health Issue

Medical staff can:

- Record injuries
- Update treatment information

This process includes Risk Analysis.

View Injury Reports

Allows medical staff to:

- Review medical records
- Monitor athlete health conditions

5. Risk Analysis

The Risk Analysis use case analyzes:

- Athlete performance
- Injury risks
- Fatigue levels
- Health conditions

It receives data from:

- Monitor Athlete Performance
- Report Health Issue
- Club Management

If serious risks are detected, the system extends to Medical Escalation.

6. Medical Escalation

Medical Escalation is triggered when:

- Severe injury risk is detected
- Health condition becomes critical

This process ensures urgent medical attention for athletes.

7. Admin

The Admin manages system access and users.

Login / Authentication

Allows secure system access.

Manage Users & Roles

Admins can:

- Create user accounts
- Assign roles
- Manage permissions

Include and Extend Relationships

Include Relationship

An include relationship means a use case always requires another process.

Examples:

- Log Training Session → Calculate Workload
- Report Health Issue → Risk Analysis
- Manage Club → Club Management

Extend Relationship

An extend relationship means an optional or conditional action may occur.

Examples:

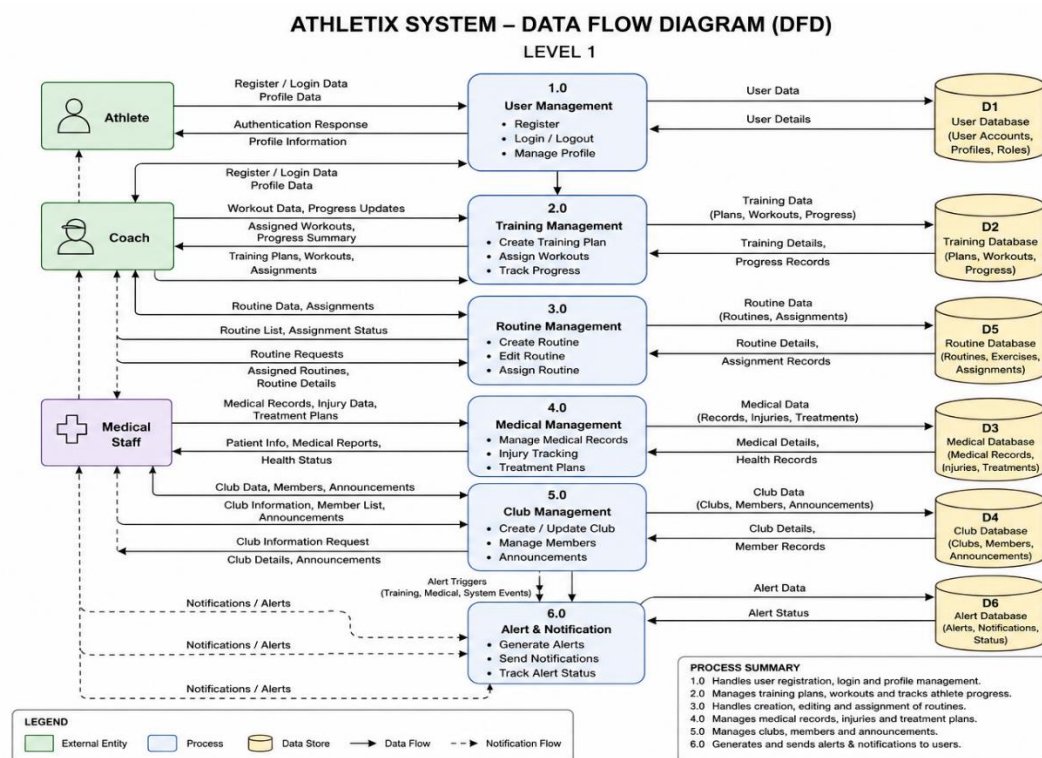
- Monitor Athlete Performance → Risk Analysis
- Risk Analysis → Medical Escalation

Overall Workflow

1. Users log into the system.
2. Athletes log training sessions and monitor progress.
3. Coaches create training plans and manage clubs.

4. Medical staff handle injuries and health reports.
5. The system performs workload calculations and risk analysis.
6. Club Management handles members, announcements, and events.
7. If severe risk is detected, Medical Escalation is triggered.
8. Admin controls user accounts and system permissions.

4.3 Data Flow Diagram (DFD)



The Level 1 Data Flow Diagram (DFD) of the ATHLETIX System explains how data flows between users, processes, and databases within the system. It breaks the system into several functional modules for better understanding.

The system mainly interacts with three external entities:

- Athlete
- Coach
- Medical Staff

These users communicate with different system processes to manage training, routines, medical records, clubs, and notifications.

1.0 The User Management process handles:

- User registration
- Login and logout
- Authentication
- Profile management

Athletes, coaches, and medical staff send login credentials and profile data to this process. The system verifies the information using the D1 User Database and returns authentication responses and user details.

This module ensures secure access control for all users.

2.0 Training Management

The Training Management process manages athlete training activities.

The coach creates:

- Training plans
- Workout schedules
- Progress updates

Athletes can access assigned workouts and monitor their progress. The process stores all training-related data in the D2 Training Database.

This module helps track athlete performance and training progress efficiently.

3.0 Routine Management

The Routine Management process is responsible for:

- Creating routines
- Editing routines
- Assigning routines to athletes

Coaches send routine and assignment data to the system, while athletes request routine details. All routine-related information is stored in the D5 Routine Database.

This module helps organize exercise schedules and workout routines.

4.0 Medical Management

The Medical Management process handles athlete health monitoring.

Medical staff upload:

- Medical records
- Injury information
- Treatment plans

The system stores this information in the D3 Medical Database. Athletes and medical staff can access health reports and recovery information when necessary.

This module supports injury tracking and athlete recovery management.

5.0 Club Management

The Club Management process manages all club-related activities.

It handles:

- Club creation
- Member management
- Club announcements
- Club information updates

Coaches manage clubs and announcements, while athletes can view club details and updates. All club information is stored in the D4 Club Database.

This module improves communication and coordination among athletes and coaches.

6.0 Alert & Notification

The Alert & Notification process generates alerts and notifications based on system activities.

Alerts may include:

- Injury warnings
- New routine assignments
- Training reminders
- Club announcements
- System notifications

The process receives alert triggers from other modules and stores alert data in the D6 Alert Database before sending notifications to users.

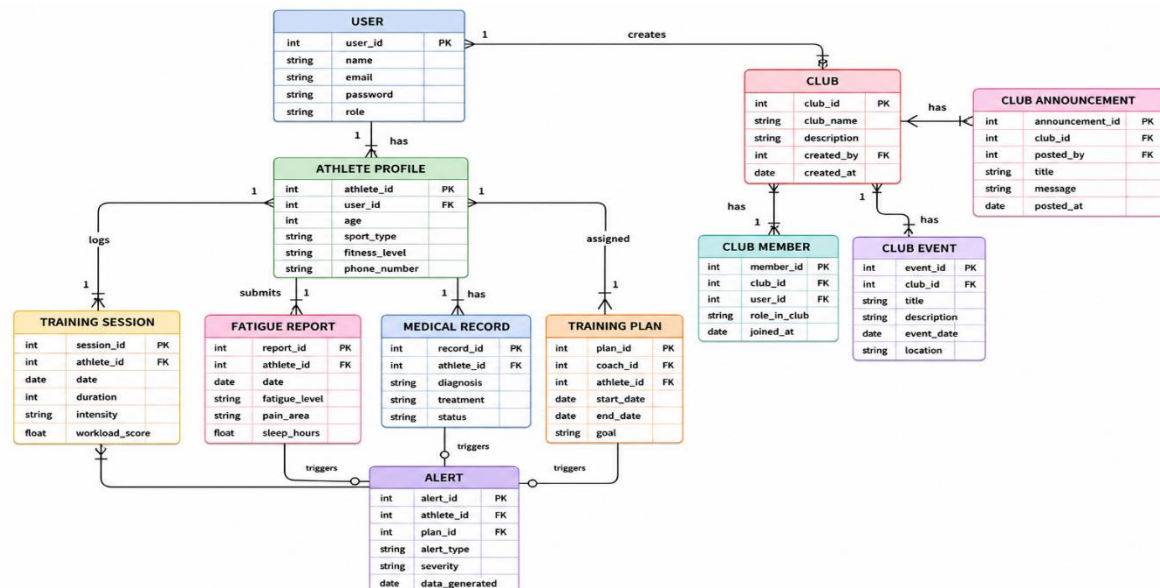
This module ensures real-time communication within the system.

Overall System Workflow

1. Users log into the system through User Management.
2. Coaches create training plans and routines.
3. Athletes access workouts, routines, and club information.
4. Medical staff manage health records and treatment plans.
5. Club Management handles club operations and announcements.
6. All information is stored in respective databases.
7. Alert & Notification sends important updates to users.

Overall, the Level 1 DFD clearly demonstrates how the ATHLETIX System processes, stores, and transfers data between users and system modules efficiently.

4.4 Database Design (ER Diagram)



The ER (Entity Relationship) Diagram represents the database structure of the ATHLETIX System. It shows the entities (tables), their attributes, primary keys (PK), foreign keys (FK), and the relationships between different modules of the system.

The system is mainly designed for:

- Athlete management
- Training management
- Medical monitoring
- Alert generation
- Club management

1. USER Entity

The USER table stores information about all system users.

Attributes

- user_id (PK) – Unique user ID
- name – User name
- email – User email
- password – User password
- role – User role (Athlete, Coach, Medical Staff, Admin)

Relationship

- One USER has one ATHLETEPROFILE
- One USER can create many TRAININGPLANS
- One USER can create CLUBs

2. ATHLETE PROFILE Entity

The ATHLETEPROFILE table stores athlete-specific information.

Attributes

- athlete_id (PK)
- user_id (FK)
- age
- sport_type
- fitness_level

- phone_number

Relationship

An athlete profile:

- Logs many TRAININGSESSIONs
- Submits many FATIGUEREPORTs
- Has many MEDICALRECORDs
- Is assigned many TRAININGPLANs

3. TRAINING SESSION Entity

This table stores athlete workout session data.

Attributes

- session_id (PK)
- athlete_id (FK)
- date
- duration
- intensity
- workload_score

Purpose

Used to:

- Track workouts
- Measure athlete workload
- Monitor performance

Relationship

Training sessions can trigger ALERTs.

4. FATIGUE REPORT Entity

The FATIGUEREPORT table stores athlete fatigue and recovery information.

Attributes

- report_id (PK)
- athlete_id (FK)
- date
- fatigue_level
- pain_area
- sleep_hours

Purpose

Used to:

- Monitor athlete fatigue
- Detect recovery problems
- Support risk analysis

5. MEDICAL RECORD Entity

This entity stores athlete medical information.

Attributes

- record_id (PK)
- athlete_id (FK)
- diagnosis
- treatment
- status

Purpose

Used for:

- Injury tracking
- Treatment management
- Athlete health monitoring

Relationship

Medical records may trigger ALERTs if severe conditions are detected.

6. TRAINING PLAN Entity

The TRAININGPLAN table stores training schedules assigned by coaches.

Attributes

- plan_id (PK)
- coach_id (FK)
- athlete_id (FK)
- start_date
- end_date
- goal

Purpose

Used to:

- Assign workouts
- Manage athlete goals
- Schedule training activities

Relationship

Training plans can trigger ALERTs.

7. ALERT Entity

The ALERT table stores generated alerts and notifications.

Attributes

- alert_id (PK)
- athlete_id (FK)
- plan_id (FK)
- alert_type
- severity
- data_generated

Purpose

Used to:

- Notify athletes and coaches
- Warn about fatigue or injury
- Generate system alerts

8. CLUB Entity

The CLUB table stores club information.

Attributes

- club_id (PK)
- club_name
- description
- created_by (FK)
- created_at

Purpose

Used to:

- Manage clubs or teams
- Organize athletes and coaches
- Store club details

Relationship

A CLUB:

- Has many CLUBMEMBERS
- Has many CLUBANNOUNCEMENTS
- Has many CLUBEVENTS

9. CLUB MEMBER Entity

This table stores club member information.

Attributes

- member_id (PK)
- club_id (FK)
- user_id (FK)
- role_in_club
- joined_at

Purpose

Used to:

- Add athletes/coaches to clubs
- Manage member roles

10. CLUBANNOUNCEMENT Entity

Stores announcements posted inside clubs.

Attributes

- announcement_id (PK)
- club_id (FK)
- posted_by (FK)
- title
- message
- posted_at

Purpose

Used for:

- Club notifications
- Important updates
- Communication

11. CLUBEVENT Entity

Stores club event information.

Attributes

- event_id (PK)

- club_id (FK)
- title
- description
- event_date
- location

Purpose

Used to:

- Schedule club activities
- Organize events and competitions

Overall System Workflow

1. Users register and log into the system.
2. Athlete profiles store athlete information.
3. Athletes log training sessions and fatigue reports.
4. Coaches create training plans.
5. Medical staff manage medical records.
6. Alerts are generated based on athlete conditions.
7. Clubs manage members, announcements, and events.

Overall, the ER Diagram demonstrates how all database entities are connected to support training management, health monitoring, alert generation, and club management within the ATHLETIX System.

5. Implementation Plan

5.1 Module Breakdown

5.1.1 Project Root & Configuration Module (manage.py, Athletix/)

- Core Structural Files: manage.py, settings.py, urls.py, asgi.py, wsgi.py.
- System Responsibility: Handles the global configuration layer of the platform, including installed application indexing, custom security middleware registration, relational database routing parameters, and static/media file directory binding. It serves as the project-level URL routing root and exposes the unified standard WSGI and ASGI application execution entry points.

5.1.2 User Application Module (user/)

- Core Structural Files: models.py, forms.py, views.py, urls.py, tests.py, migrations/.
- System Responsibility: Manages identity authentication infrastructure, user registration routines, and profile type initialization (Player, Coach, and Medical Staff profiles). Controls system access lifecycles, administrative account approval sequences, password restoration tasks, and profile media assets (profile_photos/) validation via integrated file system pipelines.
- Architectural Elements: Implements custom user and sub-profile relational tables, role-based access control (RBAC) authorization group mappings, password cryptographic processing, and structural migrations capturing data attributes such as contact vectors, user addresses, and identity flags.

5.1.3 Player Performance Application Module (player/)

- Core Structural Files: models.py, views.py, urls.py, performance_utils.py, tests.py, migrations/.
- System Responsibility: Controls core physical preparation frameworks, data models, and metrics logging infrastructure. Tracks day-to-day assigned exercises, physical workouts, and execution parameters while processing student performance histories.
- Architectural Elements: Embeds automated data aggregation scripts and specialized physiological logging systems to compile workout completions, calculate structural fitness scores, and translate raw data streams into formatted tables for analytical interfaces.

5.1.4 Coaching Operation Module (coach/)

- Core Structural Files: models.py, views.py, urls.py, tests.py.

- **System Responsibility:** Provides specific operational tools for authorized athletic mentors to construct unified training routines, assign training tracks across target schedules, and review physical progression tables for assigned student groups.
- **Architectural Elements:** Integrates cross-app relational bridges with the player module to deploy training assignments and links with the chat module to manage feedback communication channels.

5.1.5 Confidential Medical Application Module (medical_staff/)

- **Core Structural Files:** models.py, forms.py, views.py, urls.py, migrations/.
- **System Responsibility:** Establishes a protected system partition that isolates confidential student-athlete clinical data records, consultation updates, and health logs away from unauthorized accounts.
- **Architectural Elements:** Manages athletic activity clearance flag workflows (Fully Cleared, Restricted, Sidelined), handles secure binary diagnostic document uploads, and executes database-level audit log interceptors that track all data access patterns and file edits to safeguard sensitive Personally Identifiable Information (PII).

5.1.6 Contextual Communication Module (chat/)

- **Core Structural Files:** models.py (Conversation, Message), views.py, urls.py, tests.py.
- **System Responsibility:** Drives an internal message exchange platform enabling threaded discussions and multimedia document attachments among verified application users.
- **Architectural Elements:** Backs text data distribution using HTTP polling protocols or standard asynchronous WebSocket channels. The schema ensures data accountability by explicitly binding message transactions to corresponding user accounts and allowing messages to be anchored to a system context, such as a specific training routine or a physical log.

5.1.7 Administrative Control Module (Admin/)

- **Core Structural Files:** admin.py, views.py, urls.py, middleware.py.
- **System Responsibility:** Dispatches an administrative workspace console that grants superusers full control over user profiles, systemic configurations, operational logging review, and platform health telemetry.
- **Architectural Elements:** Processes user lifecycle enforcement sequences (including manual account verification approval, suspension, or deletion) and compiles system-wide administrative reports across active user clusters.

5.1.8 Presentation & Resource Assets Module (Templates/ & Static/)

- Core Structural Files: Top-level global template layouts (base.html, home.html), app-specific layout directories, and cascading asset trees.
- System Responsibility: Directs client-side user interface construction, template composition, and frontend component delivery. The module organizes globally shared layouts, design systems, and image sets used uniformly across browser viewports.

5.1.9 Binary Data Assets Module (media/)

- Core Structural Files: Localized physical disk storage arrays, including profile_photos/ and medical attachment directories.
- System Responsibility: Maintains secure persistence of user-submitted binary artifacts, diagnostic scan image data, and physical document files. It provides access control routes that serve authorized media assets safely through temporary, validated system operations.

5.1.10 System Migration & Schema History Module (migrations/)

- Core Structural Files: Ordered migration scripts distributed across all internal application structures.
- System Responsibility: Documents the developmental evolution of the database architecture, translating abstract Python ORM model definitions into precise SQL schema declarations. It maintains structural audit history data to enable reliable data patching, backward compatibility, and upgrade tracking.

5.1.11 Quality Assurance & Automated Testing Module (tests.py)

- Core Structural Files: Programmatic unit test blocks, integration scripts, and automated validation assets (selenium_screenshots/).
- System Responsibility: Executes validation checks across core functional paths using the native Django test runner or the pytest engine. The module leverages embedded automated Selenium WebDriver configurations to simulate real user actions and evaluate interface behaviors before code modifications are merged.

5.1.12 Root Utilities & System Files

- Core Structural Files: README.md, requirements.txt, generate_tests.py, db.sqlite3.
- System Responsibility: Provisions fundamental environment documentation, external software dependency lock files, and testing generation support scripts. The tracking database state is preserved locally via a standalone file system for initial verification routines.

5.1.13 Relational Data Model Architecture (Conceptual)

- **Identity to Profile Mapping:** Every parent User record maps directly to a matching profile extension via a strict one-to-one relationship (AthleteProfile, CoachProfile, or MedicalProfile).
- **Training Workflow Mapping:** A Coach builds a reusable TrainingRoutine. This template maps to an Athlete assignment record, which prompts the user to populate a PerformanceEntry to log metrics.
- **Health Record Mapping:** An Athlete entity shares a one-to-many relationship with the restricted MedicalRecord table. Changes to these medical models trigger the system's underlying security audit logger.
- **Messaging Mapping:** A Conversation tracks group membership mappings between multiple participant IDs and links entries directly to system instances, such as a targeted fitness tracker or user profile context.

5.1.14 API Architecture & Routing Ingestion Ports

- **Routing Path Allocation:** System network queries route initially through the global urls.py entry point, which delegates path matching down to individual application controllers.
- **View Processing Layer:** API connections, data controllers, and operational logic run through Class-Based Views (CBVs) or specialized Function-Based Views (FBVs) inside the respective application views.py layers to handle specific data payloads.

5.1.15 System Maintenance & Operational Guidelines

- **Development Operations:** Standard developer environments run via the localized python manage.py runserver network port, utilizing the standalone db.sqlite3 file model and serving media directly from secure system folders.
- **Production Operations:** Standard production deployments run via enterprise WSGI/ASGI application layers behind a reverse proxy container. Structural tables are maintained using a separate relational database cluster (MySQL), communications are isolated via HTTPS transport protection, and profile media handling is routed to dedicated object storage.

5.2 Iterative Development Plan

5.2 Iterative Development Plan

The ATHLETIX System was developed using an iterative and sprint-based development approach. Each sprint focused on implementing and testing specific functionalities of the system. The iterative process allowed continuous improvement, debugging, feature enhancement, and feedback integration throughout the project lifecycle.

Sprint 1: Authentication and Core Setup

Timeline: March 11, 2026 – April 08, 2026

Implemented Features

- User Login
- User Signup
- Role-Based Access Control
- Forgot Password Functionality
- Athlete Profile Management
- Initial System Testing

Outcome

The foundational structure of the ATHLETIX System was established successfully. Authentication and user management functionalities were implemented and tested.

Sprint 2: Training & Performance Management

Timeline: April 03, 2026 – April 08, 2026

Implemented Features

- Training Session Logging
- Training Level Classification
- Performance Data Tracking
- Workout Monitoring
- Performance Visualization using Charts
- Testing and Validation

Outcome

Athlete training and performance monitoring functionalities were integrated successfully using Chart.js and backend performance tracking modules.

Sprint 3: System Administration & Maintenance

Timeline: April 09, 2026 – April 14, 2026

Implemented Features

- User Management
- Data Backup & Recovery
- System Maintenance Features
- Integration Testing
- User Acceptance Testing

Outcome

Administrative functionalities and maintenance operations were completed successfully to improve system management and reliability.

Sprint 4: Alerts & Communication & Dashboards

Timeline: April 15, 2026 – April 21, 2026

Implemented Features

- Notification and Alert System
- Medical Feedback System
- Coach Dashboard
- Medical Dashboard
- Alert Visualization
- Dashboard Testing

Outcome

Communication and dashboard modules were integrated successfully to support monitoring, notifications, and user interaction.

Sprint 5: Recovery & Injury Monitoring

Timeline: April 22, 2026 – April 28, 2026

Implemented Features

- Recovery and Fitness Monitoring
- Injury History Recording
- Overtraining and Risk Detection
- Fatigue Report Management
- Recovery Data Analysis
- System Testing

Outcome

The system successfully supported injury tracking, fatigue analysis, and athlete recovery monitoring functionalities.

Week 6: Coaching History Update

Timeline: April 29, 2026 – May 05, 2026

Implemented Features

- Coaching History Management
- Coach Activity Tracking
- Historical Training Records
- Coach Performance Logs

Outcome

The coaching history module improved athlete-coach performance tracking and historical activity management.

Week 7: Club Work Update

Timeline: May 06, 2026 – May 12, 2026

Implemented Features

- Club Creation and Management
- Club Member Management

- Club Announcements
- Club Event Scheduling
- Club Activity Monitoring

Outcome

The ATHLETIX System successfully integrated a complete club management system for athletes and coaches.

5.3 Weekly Lab Session Work Distribution

The ATHLETIX System development work was divided across weekly lab sessions using a sprint-based iterative development process. Each team member was assigned specific tasks during every sprint.

Week	Sprint / Module	Assigned Person(s)	Lab Session Activities
Week 1	Authentication and Core Setup	Sunandan, Sakif, Minhaz, Rokon	Developed login, signup, role-based access control, forgot password system, athlete profile management, and performed initial testing.
Week 2	Training & Performance Management	Sunandan, Rokon	Implemented training session logging, training level classification, performance tracking, workout monitoring, chart visualization, and testing.
Week 3	System Administration & Maintenance	Sunandan	Developed user management, backup and recovery system, maintenance features, integration testing, and user acceptance testing.
Week 4	Alerts & Communication & Dashboards	Sunandan, Rokon	Implemented notification and alert system, medical feedback system, coach dashboard, medical dashboard, and dashboard testing.
Week 5	Recovery & Injury Monitoring	Sunandan, Sakif, Rokon	Developed recovery monitoring, injury history recording, overtraining

			detection, fatigue analysis, and system testing.
Week 6	Coaching History Update	Sunandan	Added coaching history management, coach activity tracking, historical training records, and performance log updates.
Week 7	Club Work Update	Sunandan	Implemented club creation, member management, club announcements, club event scheduling, and club activity monitoring.

5.3 Technology Stack

Layer	Technology / Tools	Purpose	Version
Frontend	HTML5	Structure and content design of the web application	Latest
	CSS3	Styling, layout, and responsive user interface design	Latest
	JavaScript	Interactive functionalities and dynamic behavior	Latest
	Chart.js	Data visualization and graphical performance analysis	Latest
Backend	Python Django	Server-side logic, authentication, and application management	Latest
Database	MySQL / SQLite	Data storage, retrieval, and database management	Latest
Image Processing	Pillow (PIL)	Image uploading, processing, and profile image handling	Latest
Infrastructure / Platform	Windows	Development and deployment environment	Latest

5.4 Development Methodology

The **Athletix** platform is engineered utilizing the **Agile Development Methodology**, specifically applying the **Scrum Framework**. This approach structures the software lifecycle into iterative development cycles that allow the team to continuously refine functional modules, respond immediately to structural changes in requirements, and deliver incremental software versions without breaking core dependencies.

5.4.1 The Scrum Framework Application

To maintain high development velocity and manage cross-functional dependencies across the system's Django applications (user, player, medical_staff, and chat), the project execution follows a structured, repetitive cycle of Scrum ceremonies:

- **Product Backlog & User Story Refinement:** All functional elements (from user account registration up to multi-variable progress dashboards and secure clinical data handling) are broken down into discrete User Stories. These are estimated and ordered based on system architecture priorities.
- **Sprint Planning:** At the beginning of each iterative development increment (Sprint), the development team commits to a sub-set of high-priority backlog issues, establishing a clear Sprint Goal for the upcoming cycle.
- **Daily Stand-up Meetings:** Brief daily alignment sessions are conducted to synchronize active tasks, evaluate current progress against the Sprint criteria, and immediately identify development bottlenecks or resource blockers.
- **Sprint Review & Increment Delivery:** At the conclusion of each Sprint, the functioning product increment is demonstrated. New modules (such as Chart.js rendering pipelines or Pillow image parsing validation) are reviewed against the acceptance criteria.
- **Sprint Retrospective:** The team analyzes the execution efficiency of the concluded cycle, refining internal testing strategies, code review workflows, and configuration pipelines for subsequent iterations.

5.4.2 Project Management Tooling: Jira

Project tracking, requirement traceability, and workflow transparency are maintained through **Jira** as the primary enterprise project management engine. The system infrastructure is configured to map the Scrum lifecycle explicitly:

- **Epics & Component Architecture:** High-level project domains are mapped into structural Epics within Jira (e.g., *Epic: Role-Based Access Control*, *Epic: Confidential Medical System*, *Epic: Performance Visualization*). Issues are tagged by Django application components to guarantee modular tracking.

- **Sprint Boards & Backlog Grooming:** The Jira Scrum Board provides real-time visualization of task states across standard operational columns: To Do, In Progress, Code Review / QA Testing, and Done.
- **Velocity & Burn-down Analytics:** Jira's automated telemetry engines capture Sprint Velocity and generate Burn-down charts. These data logs allow the project manager to track developer capacity, estimate downstream release windows accurately, and verify that automated testing intervals (via Selenium) align with delivery deadlines.
- **Task Allocation & Accountability:** Each functional requirement (FR-1 to FR-20) and non-functional metric is instantiated as an independent issue containing detailed descriptions, strict acceptance parameters, and direct team role assignments. This ensures continuous transparency across system analysts, programmers, architects, and QA engineers.

5.4.3 Github: <https://github.com/Sunandan-FX/Athletix-Software>

Survey: https://docs.google.com/spreadsheets/d/1IkVM8-ucLEPG_TINheroYyf3FJm2d8mQUUKZgeRY5I4/edit?resourcekey=&gid=1626203414#gid=1626203414

Jira

Project: <https://sakif23101219.atlassian.net/jira/software/projects/SCRUM/boards/3/backlog?epics=visible>

6. Development Process

6.1 Week-wise Progress

Week	Tasks	Completed Evidence (Code/Output)	Remarks
Week 1	Software Requirements Specification (SRS) Documentation	Completed SRS document including requirements analysis, Use Case Diagram, DFD, and ER Diagram	Requirement analysis completed successfully
Week 2	Software Design Specification (SDS) Documentation	Completed system architecture, database schema, module design, and technology stack documentation	Design phase reviewed and finalized
Week 3	Figma Prototype Design	Completed UI/UX prototype for athlete, coach, medical staff, and admin panels	Prototype approved after review
Week 4	Sprint 1 – Authentication and Core Setup	Implemented login, signup, role-based access control, forgot password, and athlete profile management	Initial backend and authentication system completed
Week 5	Sprint 2 – Training & Performance Management	Implemented training session logging, workout monitoring, performance tracking, and chart visualization	Training and performance module completed
Week 6	Sprint 3 – System Administration & Maintenance	Developed user management, backup & recovery system, maintenance features, and integration testing	Administrative module completed
Week 7	Sprint 4 – Alerts & Communication & Dashboards	Implemented notification system, dashboards, medical feedback, and alert visualization	Dashboard and communication module completed
Week 8	Sprint 5 – Recovery & Injury Monitoring	Developed fatigue analysis, injury history, recovery monitoring, and risk detection system	Recovery and monitoring module completed

Week 9	Coding Implementation & Test Plan Submission	Completed backend coding, module integration, unit testing, and submitted test plan documentation	System testing and coding phase finalized
Week 10	Club System Development	Implemented club creation, member management, announcements, event scheduling, and activity monitoring	Club management module completed
Week 11	Final Report Submission	Submitted final report, updated documentation, screenshots, testing results, and presentation materials	Final project submission completed successfully

6.2 Coding Implementation

The ATHLETIX System was developed using the Django framework following a modular and scalable architecture. The implementation focuses on athlete performance monitoring, training management, medical support, communication, and administrative control. The system is divided into multiple applications where each module handles a specific functionality of the platform.

Project Structure and Core Modules

The project is organized into several Django applications to ensure maintainability, code reusability, and modular development.

1. Project Configuration Module

The root configuration files include:

- manage.py
- settings.py
- urls.py
- asgi.py
- wsgi.py

These files manage:

- Global project configuration
- URL routing
- Middleware setup
- Database connectivity
- ASGI/WSGI deployment configuration

The `urls.py` file controls the routing system by connecting all application endpoints within the project.

2. Authentication and User Profile Module

The authentication module is implemented inside the **user app** using:

- `models.py`
- `views.py`
- `forms.py`

This module handles:

- User registration
- User login and logout
- Password management
- Role-based access control
- Profile management

Different user roles are implemented such as:

- Athlete
- Coach
- Medical Staff
- Admin

Custom profile models including `AthleteProfile`, `CoachProfile`, and `MedicalProfile` store role-specific information.

3. Player and Training Management Module

The **player app** manages athlete training and performance-related functionalities.

Main files:

- models.py
- views.py
- performance_utils.py

Key entities include:

- DailyRoutine
- AthleteCoach
- Sport
- Club

This module supports:

- Routine creation and management
- Athlete-coach relationship management
- Workout tracking
- Performance analytics
- Club-related activities

Performance analytics utilities are implemented to generate charts, progress summaries, and performance reports.

4. Coach Management Module

The **coach app** provides coach-specific functionalities.

This module includes:

- Coach dashboards
- Routine management
- Athlete monitoring
- Performance search

- Approval workflows

A custom `coach_required` decorator is implemented to restrict access to coach-only features. Coaches can create, edit, delete, and approve routines assigned to athletes.

5. Medical Management Module

The **medical_staff app** handles athlete medical information and health monitoring.

Key models:

- AthleteHealthRecord
- MedicalFeedback

This module allows medical staff to:

- Store medical records
- Track injuries
- Provide feedback
- Monitor athlete recovery status

The system also maintains review-friendly and audit-focused medical records for secure tracking.

6. Chat and Communication Module

The **chat app** enables communication between users.

Main features:

- Conversation management
- Real-time messaging
- Message history
- Unread message counting

Important models:

- Conversation
- Message

The system supports message persistence and last-message retrieval for communication efficiency.

7. Admin Management Module

The **Admin app** manages administrative functionalities.

Features include:

- User approval
- User deletion
- Dashboard management
- Audit-related controls

The admin panel helps maintain secure system control and user monitoring.

Important Algorithms and Functionalities

1. Performance Aggregation Algorithm

Implemented in:

- `player/performance_utils.py`

Purpose:

- Generate athlete performance metrics
- Prepare chart data
- Calculate workout statistics

Main Functionalities

- Calculate workout duration
- Compute completion rates
- Generate weekly performance summaries
- Create sport distribution statistics

The algorithm uses counters and date-based grouping to process athlete routine data efficiently.

2. Coach Search and Analytics

Implemented in:

- coach/views.py

Purpose:

- Search coached athletes
- Display performance summaries
- Generate inline analytics

The system filters athlete-coach relationships using Django ORM queries and displays analytical summaries for coaches.

3. Routine Lifecycle Management

Purpose:

- Track athlete routine completion
- Manage coach approvals

Important fields:

- athlete_marked_complete
- coach_approved_completion
- completion_message

This workflow ensures proper monitoring and approval of athlete training activities.

4. Role-Based Access Control (RBAC)

Custom decorators are implemented for role management.

Examples:

- coach_required
- admin_app_required

Purpose:

- Restrict unauthorized access
- Protect sensitive system features
- Ensure secure user permissions

These decorators verify user roles before allowing access to protected routes.

5. Chat Helper Functions

Implemented in:

- chat/models.py

Functions include:

- Last message retrieval
- Unread message counting
- Chronological message ordering

These helper methods improve chat system performance and user experience.

Database and Performance Optimization

The system uses:

- Django ORM
- select_related
- prefetch_related

These techniques reduce database query overhead and improve system performance by avoiding unnecessary database calls.

Future Improvements

Several enhancements are planned for future versions of the ATHLETIX System, including:

- Django REST Framework API integration
- JWT-based authentication
- Real-time communication using Django Channels
- Background task processing using Celery and Redis
- Advanced audit logging
- Large-scale analytics optimization

These improvements will enhance scalability, real-time communication, and system efficiency.

Testing and Validation

The project includes testing modules to validate core functionalities such as:

- Performance data generation
- Access control validation
- Routine approval logic
- Chat unread message counting

Testing ensures system reliability, accuracy, and stable performance under different conditions.

7 Testing and Sustainability Plan

7.1 Introduction

This chapter describes the testing strategy, sustainability planning, test environment, test cases, and validation procedures of the ATHLETIX System. The testing process ensures that the system performs correctly, securely, efficiently, and reliably for athletes, coaches, medical staff, and administrators.

7.2 Test Objectives

The primary objectives of the testing process are:

Functional Objectives

- Validate user registration and login
- Verify role-based access control
- Test training routine assignment and tracking
- Validate performance analytics and dashboards
- Verify medical record updates
- Test chat and communication system
- Validate club management functionalities

Non-Functional Objectives

- Ensure security and authorization

- Improve system reliability and stability
- Validate UI responsiveness
- Ensure database consistency
- Monitor system performance and error handling

7.3 Test Strategy

7.3.1 Testing Types

Test Type	Tool / Framework	Description
Unit Testing	Django TestCase (unit test)	Tests individual components such as models, views, and forms in isolation. Uses Django test client for fast execution.
UI Testing	Selenium WebDriver	Tests templates, UI components, and frontend rendering.
Security Testing	Django TestCase	Validates role-based access control and data protection mechanisms.
Regression Testing	Django TestCase + Selenium	Ensures existing features continue working after bug fixes and updates.

7.3.2 Test Environment

Component	Technology
Operating System	Windows / Mac
Framework	Django
Programming Language	Python
Database	SQLite
Browsers	Chrome, Edge, Brave

7.3.3 Test Data Strategy

The testing process used sample test data for validation and isolation purposes.

Sample Users

- player@test.com (Player)
- coach@test.com (Coach)
- med@test.com (Medical Staff)

Inputs Used

- Valid and invalid login credentials
- Training routine data
- Health records
- Chat messages
- Club information

7.3.4 Defect Strategy

Defects were classified according to severity levels:

Severity Description

High System crash or critical failure

Medium Feature not working correctly

Low UI issue or minor functionality issue

Risk Mitigation

- Authentication validation to prevent unauthorized access
- Database consistency checks
- Role-based access control for security protection

7.4 Test Cases by Module

M1 — Users (Authentication & Profile)

This module covers:

- User registration
- Login and logout

- Profile management
- Role-based access control

TC-ID	Scenario	Input / Action	Expected Result	Type
TC-U01	Signup with valid data	Fill and submit valid form	Account created in database	Unit
TC-U02	Login with correct credentials	POST valid username/password	Session created and dashboard loads	Unit
TC-U03	Login with wrong password	POST wrong password	Error message shown	Unit
TC-S01	Logout terminates session	Click logout button	Session cleared and redirected to login	Selenium

M2 — Player Module

This module covers:

- Sport selection
- Routine management
- Performance tracking
- Player dashboard

TC-ID	Scenario	Input / Action	Expected Result	Type
TC-U10	View player health record	Open specific player record	Medical data safely retrieved	Unit
TC-S07	Update health status	Submit health changes	Changes saved and visible	Selenium
TC-U11	Load medical dashboard	Navigate to medical dashboard	Medical overview and recent updates shown	Unit
TC-S08	Update medical profile	Edit medical staff details	Profile updates successfully	Selenium

M3 — Coach Module

This module covers:

- Athlete management
- Training routine creation
- Routine updates
- Coach dashboard management

TC-ID	Scenario	Input / Action	Expected Result	Type
TC-U07	Accept athlete request	Approve pending request	Athlete linked to coach in database	Unit
TC-U08	Create new routine	Submit routine form data	Routine saved and assigned	Unit
TC-S05	Update existing routine	Edit routine data and save	UI updated successfully	Selenium
TC-U09	Load coach dashboard	Navigate to coach dashboard	Coach-specific statistics displayed	Unit
TC-S06	Update coach profile	Edit coaching credentials	Profile updated successfully	Selenium

M4 — Medical Staff Module

This module covers:

- Health record retrieval
- Health status updates
- Medical profile management

TC-ID	Scenario	Input / Action	Expected Result	Type
TC-U10	View player health record	Open specific player record	Medical data safely retrieved	Unit
TC-S07	Update health status	Submit health changes	Changes saved and visible	Selenium
TC-U11	Load medical dashboard	Navigate to medical dashboard	Medical overview and recent updates shown	Unit

TC-S08	Update medical profile	Edit medical staff details	Profile updated successfully	Selenium
--------	------------------------	----------------------------	------------------------------	----------

M5 — Chat Module

This module covers:

- Real-time messaging
- Chat history loading
- Communication reliability

TC-ID	Scenario	Input / Action	Expected Result	Type
TC-U09	Send real-time message	Type text and POST	Message saved to database	Unit
TC-S06	Load chat history	Open chat window	Previous messages loaded in UI	Selenium

M6 — Database Operations

This module covers:

- Database constraints
- Data consistency
- Cascading relationships

TC-ID	Scenario	Input / Action	Expected Result	Type
TC-U13	Model constraint validation	Save model with missing fields	Database throws Integrity Error	Unit
TC-U14	Cascading deletes	Delete coach profile	Associated routines deleted	Unit
TC-U15	Seed data generation	Run database seed command	Correct number of records populated	Unit

M7 — Admin Panel

This module covers:

- Admin authentication

- User management
- Administrative dashboard

TC-ID	Scenario	Input / Action	Expected Result	Type
TC-U16	Admin access control	Access admin panel as regular user	Access denied / HTTP 403 Forbidden	Unit
TC-S10	Admin login & dashboard	Login using superuser credentials	Admin dashboard loaded successfully	Selenium
TC-S11	Manage user records	Admin deletes user via panel	User permanently removed	Selenium

7.5 Test Coverage Summary

Module	Unit Tests	Selenium Tests	Total	Status
Authentication System	12	3	15	Passed
Player Module	10	3	13	Passed
Coach Module	10	3	13	Passed
Medical Module	8	2	10	Passed
Chat Module	6	2	8	Passed
Database Operations	10	0	10	Passed
Admin Panel	5	3	8	Passed
TOTAL	61	16	77	—

7.6 Pass / Fail Criteria

7.6.1 Pass Criteria

- All unit tests pass successfully
- Selenium tests complete without errors
- No authentication bypass occurs
- Data isolation is maintained
- Database records generate correctly

7.6.2 Fail Criteria

- Invalid HTTP responses occur
- Forms accept invalid data
- Unauthorized access becomes possible
- Selenium tests fail because of missing UI elements

7.7 Test Schedule

Phase	Activity	Deliverable
Phase 1	Write unit tests for M1–M4	test_auth.py, test_player.py, test_coach.py, test_medical.py
Phase 2	Write unit tests for M5–M7	test_chat.py, test_database.py, test_adminpanel.py
Phase 3	Write Selenium tests for all modules	selenium/test_*.py files
Phase 4	Execute all tests and fix failures	Test execution report
Phase 5	Document results and submit	Final test report

7.8 Sustainability Plan

The ATHLETIX System was designed to support long-term sustainability and maintainability.

Maintainability

- Modular Django application structure
- Reusable templates and components
- Clean separation of frontend and backend

Scalability

- Optimized database design
- Expandable role-based architecture
- Future REST API support

Security Sustainability

- Role-based access control
- Secure authentication and authorization
- Protected user data handling

Performance Sustainability

- Optimized Django ORM queries
- Use of `select_related` and `prefetch_related`
- Efficient dashboard analytics

Future Enhancements

- Django REST Framework API
- Real-time communication using Django Channels
- Advanced analytics and reporting

7.9 Tools & Dependencies

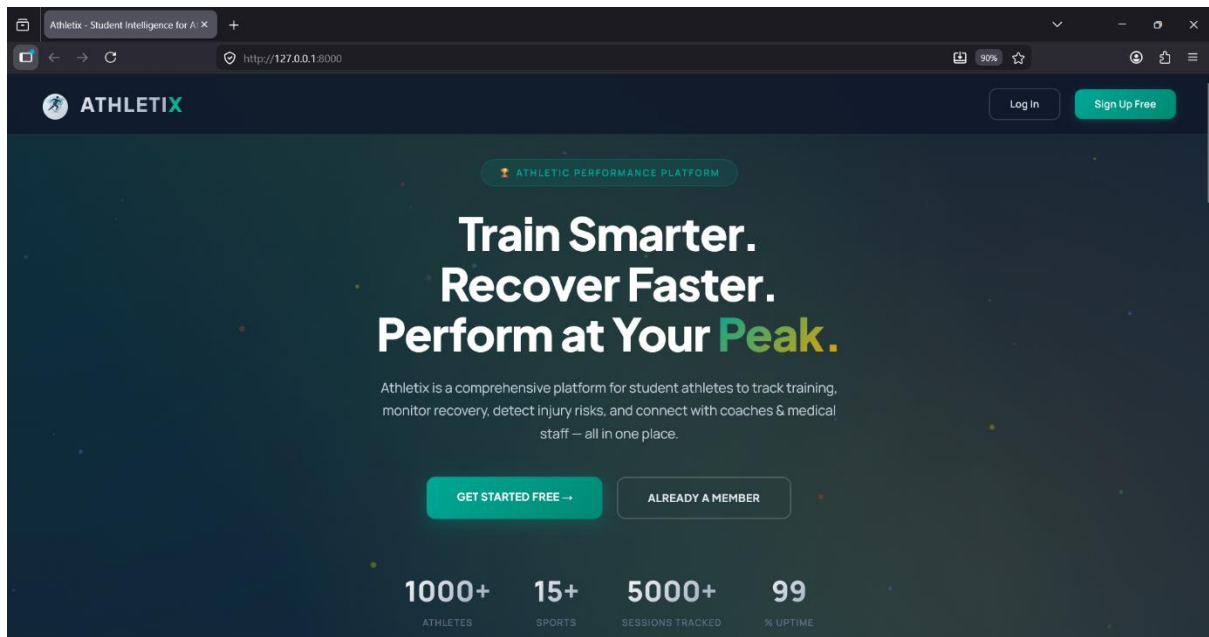
Tool	Version	Purpose
Django TestCase	Django 5.2.6	Unit Testing
Selenium	4.x	Browser automation testing
ChromeDriver	Latest (Match Chrome)	Selenium browser backend
LiveServerTestCase	Django Built-in	Local server testing for Selenium

7.10 Test Conclusion

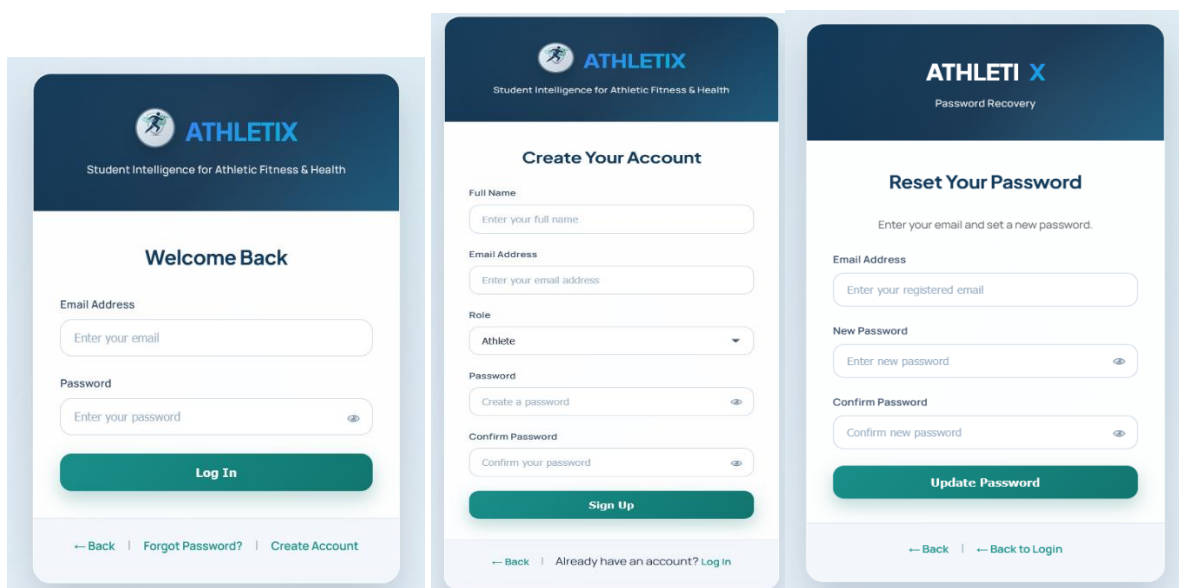
The testing and sustainability planning process ensured that the ATHLETIX System meets both functional and non-functional requirements. The testing successfully validated authentication, training management, medical monitoring, communication systems, database operations, and administrative functionalities. The sustainability plan also ensures future scalability, maintainability, security, and long-term system performance.

8. Results and Discussion

8.1 Final output screenshots:



This interface represents the homepage of the Athletix software project. Where the Project start.



Login and signup page where user login into the system of the system or create a account and if user forget his password than he can create new password.

Welcome, Hamza!

Athlete Dashboard

Back

My Profile

Search Profiles

Dashboard

Chat

Logout

Athlete Performance Hub

Track your routine, manage coaches, and share your health condition with medical staff.

 Quick Actions

Select Sports

Find a Coach

My Coaches

Coaching History

View Routines

Select Club

Performance Chart

My Profile

View Feedback

Add Health Metrics



MY SPORTS

2

Manage Sports



ACTIVE COACHES

1

View Coaches



PENDING REQUESTS

1

Find Coaches



TODAY'S ROUTINES

0

View All



MY CLUB

FireFox Badminton ...

Select Club

Select Your Sports

Choose the sports you want to train in

Back

Dashboard

Logout

AVAILABLE SPORTS

☐ Athletics
Track and field events

☒ Badminton
Racket sport using a shuttlecock

☐ Basketball
Team sport played on a rectangular court

☐ Boxing
Combat sport involving punching

☐ Cricket
Bat and ball game played between two teams

☐ Cycling
Sport using bicycles

☒ Football
Team sport played with a spherical ball

☐ Gymnastics
Sport requiring balance and flexibility

☐ Hockey
Team sport played with sticks and a ball

☐ Kabaddi
Contact team sport from India

☐ Swimming
Individual or team sport in water

☐ Table Tennis
Racket sport played on a table

☐ Tennis
Racket sport played individually or in pairs

☐ Volleyball
Team sport with a ball over a net

☐ Wrestling
Combat sport involving grappling

Save Selection

Find a Coach

Browse and request coaches for your training

[Back](#)
[Dashboard](#)
[Logout](#)

Filter by Sport:

All Sports

Filter

Z

ZICO

Coach

SPECIALIZATION

Good at attack

SPORT

Badminton

EXPERIENCE

5 years

✓ Already Your Coach

Zk

ZAM KING

Coach

SPORT

Cricket

Request Pending

GJ

GALIB JIR

Coach

SPECIALIZATION

Boxing

SPORT

Boxing

EXPERIENCE

4 years

X Sport Not Selected

Select Boxing to request this coach

My Coaches Side of Player:

My Coaches

Your assigned coaches and pending requests

Back

Find Coaches

Dashboard

✓ Active Coaches

Z

ZICO

Active Coach

Badminton

Since Apr 07, 2026

⌚ Pending Requests

Zk

ZAM KING

Pending

Badminton

Requested on Apr 08, 2026

Coaching History

See when each coach relationship started and how long it has been active

Back

My Coaches

Dashboard

Z

ZICO

Badminton

Active

Joined: Apr 07, 2026

Working for: 1 month, 2 weeks

Status: Currently training together

Daily Routine of Training :

Daily Routine

Your training schedule and routines

Back

Dashboard

Logout

Filter by Day: All Days

Filter

WEEKLY SCHEDULE

Monday

1st Day of Training

Badminton • Coach:

Complete (Coach Approved) [View Details →](#)

08:00 - 17:00

Tuesday

Moring jogging session

Badminton • Coach:
May 04, 2026

Not complete [View Details →](#)

06:00 - 20:00

Running

Badminton • Coach:

Not complete [View Details →](#)

08:00 - 17:00

Profile of the Player:

My Profile

View and manage your athlete profile

Back

Dashboard

Logout

Hamza Khan D

Athlete

O+

Edit Profile

PERSONAL INFORMATION

Edit

FULL NAME

Hamza Khan D

EMAIL

Hamza@gmail.com

PHONE

01812345632

DATE OF BIRTH

Mar 04, 2011

BLOOD GROUP

O+

MEMBER SINCE

Apr 04, 2026

ADDRESS

45/h, Niratola ,Jatra Bari,Dhaka

ATHLETE DETAILS

Edit

AGE

22

HEIGHT

5'8

WEIGHT

77

FITNESS LEVEL

Medium

CLUB

FireFox Badminton Club (Badminton)

Coach: Zico

MY SPORTS

Edit

Badminton

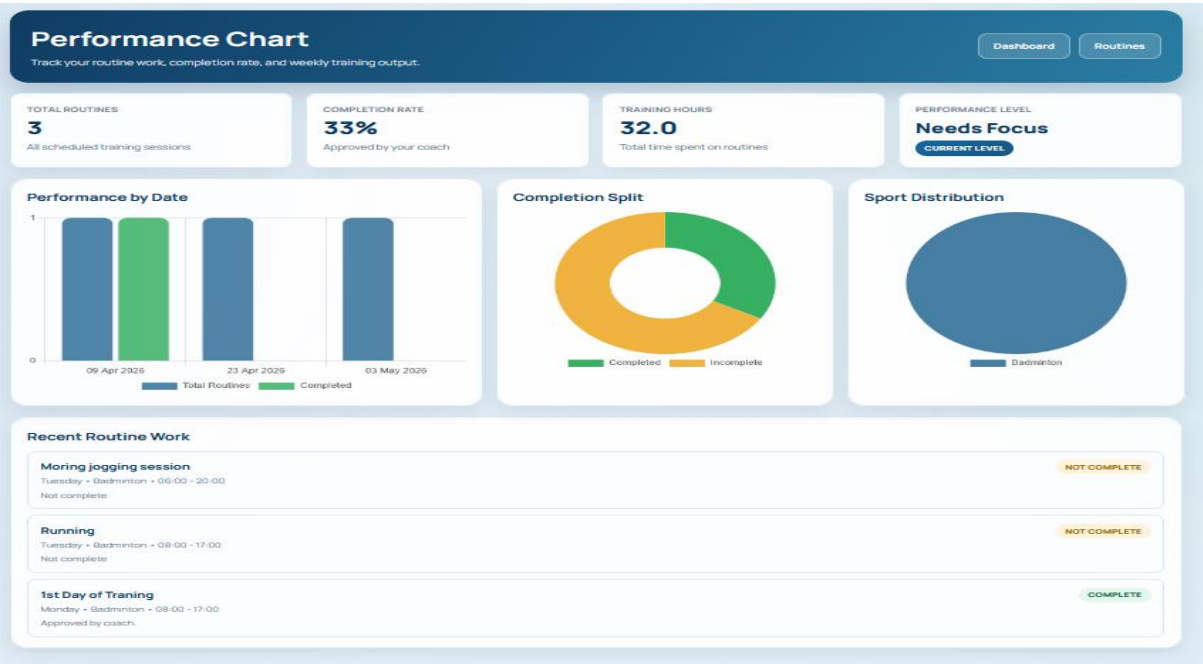
Football

MY COACHES

Find More →

Zico (Badminton)

Performance of the Player:



Player Checkup Report:

SUBMIT TODAY'S CONDITION

Your medical team will review these metrics and send feedback.
Select Doctor (optional)

Resting Heart Rate (bpm)

e.g. 64

Blood Pressure

e.g. 120/80

Weight (kg)

e.g. 68.40

Sleep Hours

e.g. 7.5

Fatigue (1-10)

1

Injury Status

No Injury

Injury Notes

Describe symptoms, pain location, or discomfort

Recovery Status

Good

Performance Notes

Energy, mobility, breathing, soreness, etc.

☐ Request medical review

Save Metrics

RECENT ENTRIES

May 01, 2026 23:49

HR 78 bpm • Fatigue 6/10 • No Injury

Needs Monitoring

Apr 28, 2026 20:05

HR 82 bpm • Fatigue 7/10 • Moderate Injury

Needs Monitoring

Recent Medical Feedback

Take Bed rest

Take full bed rest for two days without any movement

Recommendations: 1. Drink water
2. put Ointment on the part of the injury

Apr 28, 2026

Profile Search :

Profile Search

Find any user and open their full profile details.

Back

Dashboard

Logout

User Directory

Search by name, email, or role and open full profiles for athletes, coaches, and medical staff.

Search users by name, email, or role

Search

19 profiles found

A

ATHLETIX

athletix@yourcompany.com

ADMIN

View Full Profile

AA

Admin App User

admin@gmail.com

ADMIN

View Full Profile

DO

Dev One

dev1@yourcompany.com

COACH

View Full Profile

DJ

Dr. Jhon Rok

Jhon@gmail.com

DOCTOR - SPECIALTY NOT SET

View Full Profile

DV

Dr. Vir Dey

Vir@gmail.com

DOCTOR - PHYSIOOTHERAPY

View Full Profile

GJ

Galib Jir

Galib@gmail.com

COACH

View Full Profile

G

Gib

Gib@gmail.com

ATHLETE

View Full Profile

HK

Hamza Khan D

Hamza@gmail.com

ATHLETE

View Full Profile

JD

Jim Datta

Jim@gmail.com

DOCTOR - SPECIALTY NOT SET

View Full Profile

KH

Kamal Hassan

Kamal@gmail.com

ATHLETE

View Full Profile

MA

Main Admin

your-admin-email@example.com

COACH

View Full Profile

RI

Riya Islam

Riya@gmail.com

DOCTOR - SPECIALTY NOT SET

View Full Profile

RK

Rob khan

Rob@gmail.com

ATHLETE

View Full Profile

RK

Roy khan

Roy@gmail.com

COACH

View Full Profile

S

Sua

3434@gmail.com

ATHLETE

View Full Profile

S

Sunandan

Sunandan@admin.com

COACH

View Full Profile

SD

Sunandan Das

23101218@uap-bd.edu

ATHLETE

View Full Profile

ZK

Zam king

Zam@gmail.com

COACH

View Full Profile

Z

Zico

Zico@gmail.com

COACH

View Full Profile

Chat System :

Zico

COACH

← Back

Sure

Apr 29, 01:55

To maintain your good performance you have to follow some steps 1. You have see the routine and Properly finish this on time 2.Do some warmup exercise before your starting work.

Apr 29, 01:57

in this way will grow much faster and train quickly

Apr 29, 01:58

Hello coach

May 22, 02:50

Hello

May 22, 02:53

Type your message here...

Send

Dr. Vir Dey

MEDICAL STAFF

← Back

hello hamza

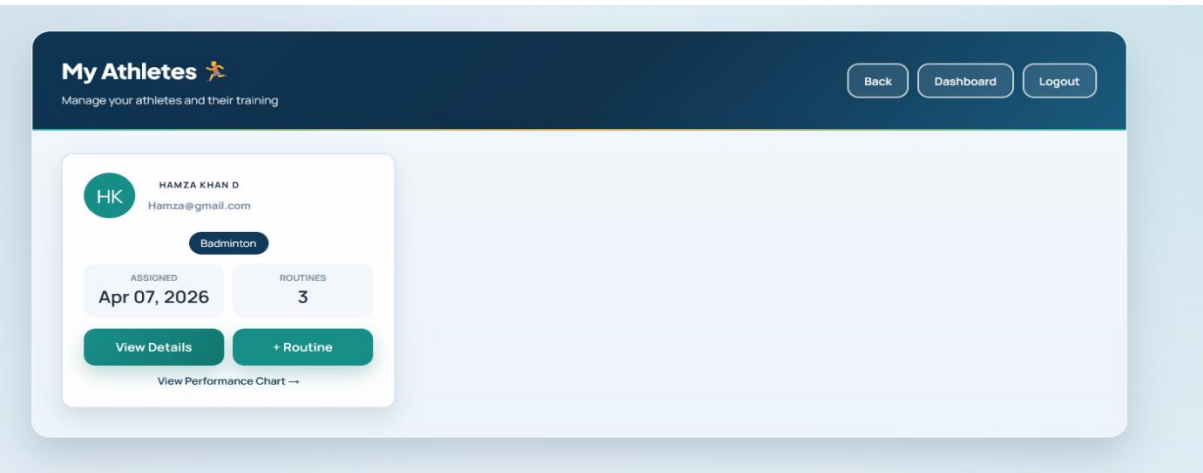
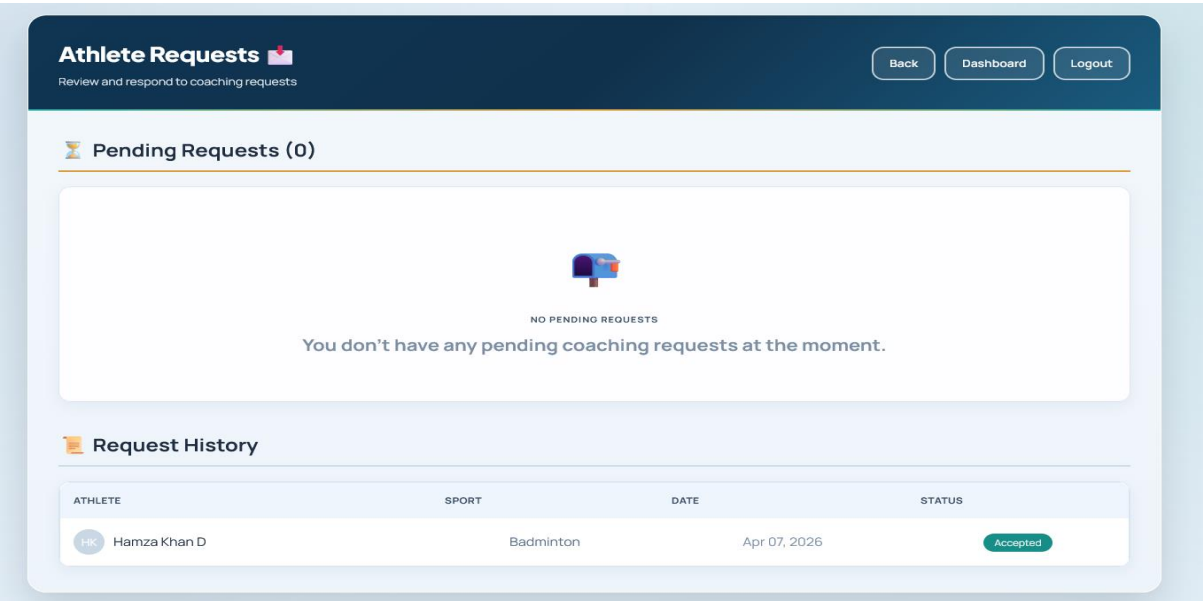
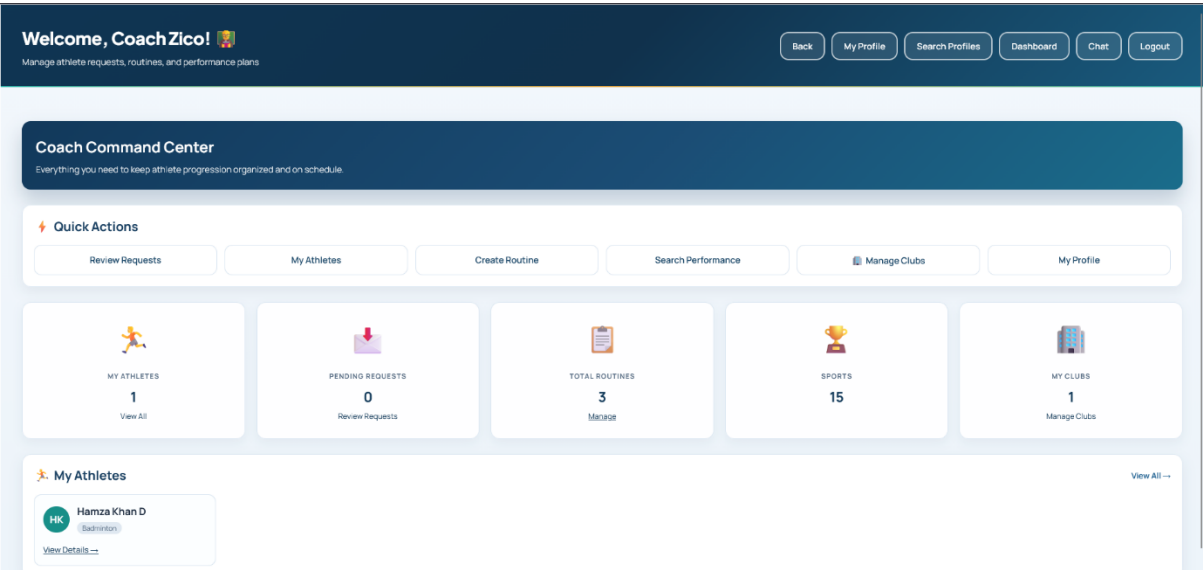
May 22, 03:00

Type your message here...

Send

Coach Side Screenshots:

Screenshot of Dashboard , Player Request, My athlete



Screenshot of Create Training routing and My club main page

Create Training Routine 

Select an athlete and add the routine details.

Back

Dashboard

Dashboard

Athlete *

Select athlete

Routine Title *

e.g., Morning Cardio Session

Sport *

Select a sport

Day of Week *

Select a day

Workout Date *

mm / dd / yyyy

Start Time *

-- : --

End Time *

-- : --

Description

Brief description of the routine...

Exercises

List the exercises, sets, reps, etc...

Notes for Athlete

Any additional notes or instructions...

Create Routine

Final Complete

Cancel

 My Clubs

Create and manage clubs for your sports

+ Create New Club

Dashboard

FIREFOX BADMINTON CLUB

Badminton

📍 York Sports Complex, Dhaka, Bangladesh

View Members

Edit

Delete

👤 1 member

📅 Created: May 13, 2026

Firefox Badminton Club is a competitive and community-focused badminton club dedicated to players of all skill levels. We provide professional coaching, regular practice sessions, fitness training, friendly matches, and tournament preparation in a supportive environment. Our club helps athletes improve footwork, reflexes, strategy, and overall performance while promoting teamwork, discipline, and sportsmanship. Whether you are a beginner learning the basics or an advanced player aiming for competitive success, SmashPoint offers the facilities and guidance to help you grow in the sport.

Medical staff Screenshot(Dashboard) :

Medical Staff

Review athletes' submitted "metrics" and provide evidence-based feedback.

Back

My Profile

Search Profiles

Dashboard

Chat

Medical Dashboard

Logout

Medical Intelligence Desk

Review complete athlete health history & provide practice recovery and performance feedback.

Ath

ACTIVE ATHLETES

6

Rec

HEALTH RECORDS

2

Fb

FEEDBACK ENTRIES

2

Alr

NEEDS MONITORING

2

Provide Health/Performance Feedback

PROVIDE HEALTH/PERFORMANCE FEEDBACK

Athlete:

Feedback type:

Title:

Feedback:

Recommendations:

Submit Feedback

Injury Status Summary

Name

1

Moderate

1

Latest Feedback

Kamal Hassan

Test

Health - May 01, 2026 02:57

Hanisa Khan D

Take Bed rest

Health - Apr 28, 2026 03:20

Complete Athlete Health Records

Completed (Feedback Given)

ATHLETE	HEART RATE	FATIGUE	INJURY	RECORDED	ACTION
Hanisa Khan D	80 bpm	7/10	Moderate Injury	Apr 28, 2026 03:05	View
Feedback history Take Bed rest Take full bed rest for two days without any movement. Recommendation: 1. Drink water 2. and Get back on the part of the injury. Apr 28, 2026					

No Feedback Yet

ATHLETE	HEART RATE	FATIGUE	INJURY	RECORDED	ACTION
Hanisa Khan D	78 bpm	6/10	No injury	May 01, 2026 23:49	Review

Hamza Khan D

Complete Health Profile & Medical History

HEART RATE (LATEST)

78 bpm

FATIGUE LEVEL

6/10

INJURY STATUS

No Injury

RECOVERY STATUS

Needs Monitoring

Health Records History (2)

May 01, 2026 • 23:49

Needs Monitoring

HEART RATE

78 bpm

BLOOD PRESSURE

122/56

WEIGHT

70.77 kg

SLEEP

5.6 hrs

FATIGUE

6/10

INJURY

No Injury

Injury Notes:

1. Head pain

Performance Notes:

1. Energy is less

🔔 Review Requested

Apr 28, 2026 • 20:05

Needs Monitoring

HEART RATE

80 bpm

BLOOD PRESSURE

124/74

WEIGHT

70.00 kg

SLEEP

5.4 hrs

FATIGUE

7/10

INJURY

Moderate Injury

Injury Notes:

1. leg injury

Performance Notes:

1. Running problem

Medical Feedback History (1)

Take Bed rest

Apr 28, 2026 • 20:20

Health Feedback

Take full bed rest for two days without any movement

Recommendations:

1. Drink water
2. put Dettol on the part of the injury

By: Dr. Vr Dey

My Profile of Doctor :

My Profile

View and manage your Medical Staff profile

Back

Dashboard

Logout

DV

Dr. Vir Dey

Doctor - Physcotherapy

B+

Edit Profile

PERSONAL INFORMATION

Edit

FULL NAME

Dr. Vir Dey

EMAIL

Vir@gmail.com

PHONE

01234567891

DATE OF BIRTH

Apr 22, 1990

BLOOD GROUP

B+

MEMBER SINCE

Apr 23, 2026

ADDRESS

Ragin road dhaka

DOCTOR SPECIALTY DETAILS

Edit

LICENSE NUMBER

24465637582546

SPECIALTY

Physcotherapy

MEDICAL ACTIVITY

Open Dashboard

HEALTH RECORDS ADDED

2

FEEDBACK ENTRIES

2

LATEST HEALTH RECORDS

Hamza Khan D • No Injury • May 01, 2026

Hamza Khan D • Moderate Injury • Apr 28, 2026

Feedback completed screenshot:

Medical Staff Dashboard

Athlete Health Profile & History

Back

Dashboard

Health Records History

May 01, 2026 • 23:49

HEART RATE

78 bpm

BLOOD PRESSURE

122/56

WEIGHT

70.77 kg

SLEEP

5.6 hrs

FATIGUE

6/10

INJURY

No Injury

Needs Monitoring

Injury Notes:

1.Head pain

Performance Notes:

1.Energy is less

Review Requested

Medical Feedback

Athlete:

Hamza Khan D (Hamza@gmail.com) - Athlete

Feedback type:

Title:

Feedback:

Recommendations:

Submit Feedback

Admin :

Admin App

Manage players, coaches, medical staff, and approvals

Back

All Users

Players

Coaches

Medical Staff

Logout

Welcome back, Admin!

PENDING COACH APPROVALS

4

TOTAL PLAYERS

7

TOTAL COACHES

7

TOTAL MEDICAL STAFF

4

PENDING MEDICAL APPROVALS

1

PENDING COACH REQUESTS

NAME	EMAIL	SPORT	SPECIALIZATION	EXPERIENCE	ACTION
Zam king	Zam@gmail.com	Cricket	Not set	0 years	Approve
Galib Jir	Galib@gmail.com	Boxing	Boxing	4 years	Approve
Sunandan	Sunandan@admin.com	Not set	Not set	0 years	Approve
Roy khan	Roy@gmail.com	Not set	Not set	0 years	Approve

PENDING MEDICAL STAFF REQUESTS

NAME	EMAIL	LICENSE NUMBER	SPECIALTY	ACTION
Jim Datta	Jim@gmail.com	Not set	Not set	Approve

All Users

View, edit, activate/deactivate, and delete users

Back

Dashboard

All Users

Players

Coaches

Medical Staff

Logout

PENDING COACH APPROVALS

NAME	EMAIL	SPORT	SPECIALIZATION	EXPERIENCE	ACTION
Roy khan	Roy@gmail.com	Not set	Not set	0 years	Approve Coach
Sunandan	Sunandan@admin.com	Not set	Not set	0 years	Approve Coach
Galib Jir	Galib@gmail.com	Boxing	Boxing	4 years	Approve Coach
Zam king	Zam@gmail.com	Cricket	Not set	0 years	Approve Coach

PENDING MEDICAL APPROVALS

NAME	EMAIL	LICENSE NUMBER	SPECIALTY	ACTION
Jim Datta	Jim@gmail.com	Not set	Not set	Approve Medical

USERS

NAME	EMAIL	ROLE	ACTIVE	APPROVED	ACTIONS
Gib	Gib@gmail.com	Athlete	Yes	Yes	Edit Deactivate Delete
Jim Datta	Jim@gmail.com	Medical Staff	Yes	No	Edit Deactivate Delete
Riya Islam	Riya@gmail.com	Medical Staff	Yes	Yes	Edit Deactivate Delete
Admin App User	admin@gmail.com	Admin	Yes	Yes	Edit Deactivate Delete
Dr. Vir Dey	Vir@gmail.com	Medical Staff	Yes	Yes	Edit Deactivate Delete
Roy khan	Roy@gmail.com	Coach	Yes	No	Edit Deactivate Delete

8.2 System performance

Project Snapshot

The ATHLETIX System is a Django-based web application where the main entry point of the project is `manage.py`. The current development database is `db.sqlite3`, which is suitable for development and testing purposes but not recommended for production deployment due to concurrency limitations.

The main project configurations are managed inside `settings.py`. Static files and uploaded media files are currently served directly from the application using the `static` and `profile_photos` directories.

Although the current implementation works efficiently for development and small-scale usage, several performance and scalability risks were identified for production-level deployment.

Current Risks and Potential Bottlenecks

1. Database Limitations

The project currently uses SQLite as the database engine. SQLite performs well during development but has limitations in handling multiple concurrent users and large-scale operations.

Potential issues include:

- Limited concurrent database writes
- Slow performance under high traffic
- N+1 query problems in multi-module views

To improve scalability, PostgreSQL is recommended for production deployment.

2. Lack of Caching System

The system currently does not use a production-level caching mechanism.

Potential impacts:

- Repeated database queries
- Increased server response time
- Higher template rendering overhead

A Redis-based caching layer can significantly improve system performance by storing frequently accessed data.

3. Static and Media File Delivery

Static assets and uploaded profile images are currently served directly from the Django application.

Potential impacts:

- Increased application server load
- Higher latency during page rendering
- Reduced scalability

Using CDN-based delivery or cloud storage solutions such as AWS S3 and CloudFront can optimize asset delivery.

4. Long Running Tasks

Some operations such as:

- Image processing
- Report generation
- Analytics calculations
- Chat processing

may execute directly during HTTP requests.

Potential impacts:

- Slow page loading
- Worker blocking
- Reduced throughput

Background task queues such as Celery with Redis can solve this issue.

5. Lack of Monitoring and Profiling

The project currently lacks monitoring and profiling tools.

Potential issues:

- Difficult to identify slow queries
- Hard to detect performance bottlenecks

- Limited debugging capabilities

Monitoring tools and profiling frameworks are necessary for performance optimization.

Immediate Improvements

The following improvements are recommended for immediate implementation.

1. Switch to PostgreSQL

For production deployment, PostgreSQL should replace SQLite.

Required Package

`pip install psycopg2-binary`

Benefits

- Better concurrency handling
- Improved scalability
- Advanced indexing support
- Better production stability

2. Add Profiling Tools

Performance profiling tools should be added during development.

Required Packages

`pip install django-debug-toolbar django-silk`

Purpose

- Identify slow database queries
- Detect N+1 query problems
- Monitor view performance
- Analyze request execution time

3. Enable Redis Caching

Redis should be used as the primary caching layer.

Required Packages

pip install redis celery

Benefits

- Faster page loading
- Reduced database load
- Improved dashboard performance
- Efficient session and cache management

4. Optimize Static File Delivery

Static assets should be collected and served using optimized deployment methods.

Recommended Solutions

- Whitenoise
- AWS S3
- CloudFront CDN

Benefits

- Reduced application server load
- Faster asset delivery
- Better scalability

Performance Profiling Process

The following profiling workflow is recommended:

1. Install django-debug-toolbar
2. Analyze SQL queries per page
3. Install django-silk
4. Profile slow views and endpoints
5. Detect N+1 database queries
6. Perform lightweight load testing using Locust

Optional Load Testing Tool

pip install locust

Short-Term Improvements

Database Optimization

- Use select_related
- Use prefetch_related
- Add indexes on frequently filtered fields

These optimizations reduce unnecessary database queries and improve query execution speed.

Background Task Processing

Heavy operations should be moved to asynchronous task queues.

Recommended Technologies

- Celery
- Redis
- Django RQ

Suitable Tasks

- Email sending
- Analytics generation
- Image processing
- Notification handling

WSGI / ASGI Optimization

The application server should be optimized using:

- Gunicorn
- Uvicorn
- Nginx

Benefits

- Better request handling
- Improved concurrency
- Load balancing support

Database Connection Pooling

Using PgBouncer with PostgreSQL can reduce connection overhead and improve database efficiency.

Long-Term Scalability Plan

Monitoring and Observability

The following tools are recommended for advanced monitoring:

Tool	Purpose
Prometheus	Metrics collection
Grafana	Performance visualization
Sentry	Error tracking
New Relic / Elastic APM	Application performance monitoring

Automated Load Testing

Regular load testing should be automated using:

- Locust
- Performance benchmarking tools

This helps identify scalability limits and performance degradation early.

Real-Time Communication Optimization

For future chat and real-time communication features, asynchronous support should be implemented using:

- Django Channels
- ASGI architecture
- WebSocket communication

This will improve real-time messaging efficiency and scalability.

8.3 Issues encountered and solutions

During the development of the ATHLETIX System, several technical and implementation-related challenges were encountered. These issues mainly occurred during chart integration, admin system configuration, club system development, and software testing. Different tools, external resources, and implementation strategies were used to solve these problems successfully.

1. Performance Chart Integration Issue

Issue Encountered

One of the major problems faced during development was creating athlete performance charts and displaying them properly on the coach dashboard. The system initially could not visualize workout progress, performance analytics, and athlete statistics effectively.

Solution

This issue was solved using the **Chart.js** library. Different chart configurations and visualization techniques were implemented to display:

- Workout progress
- Athlete performance statistics
- Training completion rates
- Dashboard analytics

Additional support was taken from GitHub source code examples and documentation to understand chart integration with Django templates and JavaScript.

Result

The coach dashboard successfully displayed interactive charts and graphical performance analytics.

2. Admin Application Configuration Issue

Issue Encountered

Another major issue occurred while creating the admin system. Initially, the system could not provide complete administrative access and permission management properly. It was difficult to configure full admin privileges and secure system control.

Solution

The issue was solved using Python PowerShell command methods and Django administrative configuration techniques. Superuser creation and role-based administrative access were configured properly through Django management commands.

Result

The system successfully supported:

- Full admin access
- User management
- Administrative dashboard control
- Secure role-based permissions

3. Club System Development Issue

Issue Encountered

While implementing the club management system, several existing modules and database relationships needed modification. Integrating club creation, member management, announcements, and event scheduling caused conflicts with previous code structures.

Solution

This problem was solved by analyzing GitHub source code examples and restructuring several parts of the project codebase. Database relationships and models were updated to support:

- Club creation
- Club member management
- Club announcements
- Club events

Result

The ATHLETIX System successfully integrated a complete club management module for athletes and coaches.

4. Testing Differentiation Issue

Issue Encountered

During software testing, there was confusion in separating Unit Tests and Selenium Tests. Initially, both testing methods were difficult to organize and execute independently.

Solution

The issue was solved by using tag functions and categorized testing structures. Separate tagging and execution methods were implemented to distinguish:

- Unit Testing
- Selenium Testing

This allowed tests to run independently without affecting each other.

Result

The testing process became more organized, efficient, and easier to maintain. Both unit tests and Selenium tests could be executed separately for better validation and debugging.

9. Conclusion

The ATHLETIX System was successfully developed as a complete sports management and athlete monitoring platform using the Django framework. The project achieved its primary objectives by implementing important features such as:

- Athlete performance tracking
- Training and routine management
- Medical monitoring and injury tracking
- Coach and medical dashboards
- Chat and communication system
- Club management and event handling
- Alert and notification system

Throughout the project, significant learning outcomes were achieved. The development process helped improve practical knowledge and technical skills in:

- Django web development
- Python programming
- Database management using SQLite
- Frontend development using HTML, CSS, JavaScript, and Chart.js
- Software testing using Unit Testing and Selenium
- Role-based authentication and system security
- System analysis and design using UML, DFD, ER diagrams, and Figma prototyping

- Debugging, problem-solving, and project management

Several technical challenges were encountered during development, including performance chart integration, admin configuration, club system implementation, and testing organization. These problems were successfully solved using external libraries, GitHub resources, Django utilities, and testing strategies.

For future improvements, the system can be enhanced by adding:

- REST API integration
- Mobile application support
- PostgreSQL database support
- Cloud deployment and advanced analytics
- Redis caching and performance optimization

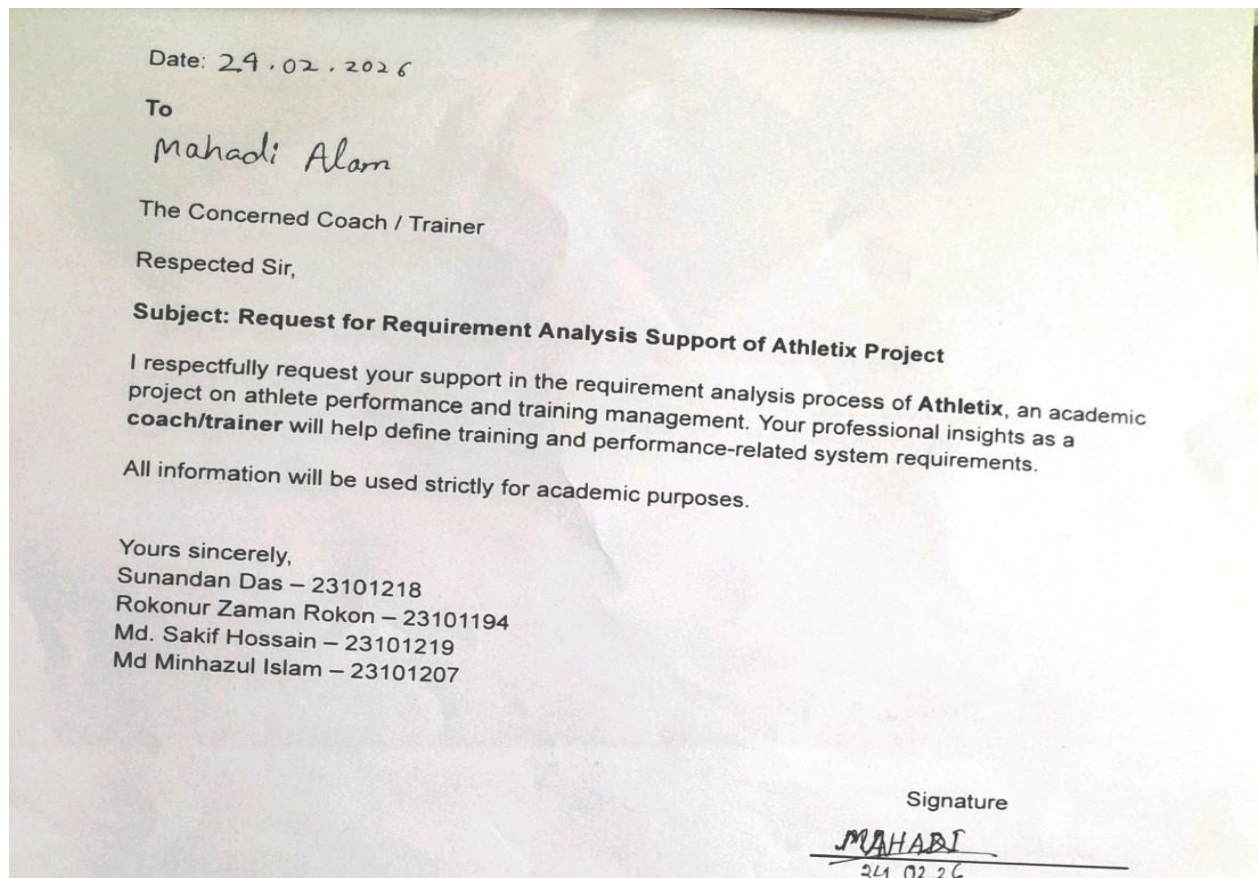
Overall, the ATHLETIX System provides a secure, scalable, and user-friendly solution for modern sports and athlete management while also serving as a valuable learning experience in full-stack software development.

Appendix:

Git link: <https://github.com/Sunandan-FX/Athletix-Software.git>



Evidence of Interaction with stakeholders:



Date: 24.02.2026

To

Kawsar Ahmed

The Concerned Coach / Trainer

Respected Sir,

Subject: Request for Requirement Analysis Support of Athletix Project

I respectfully request your support in the requirement analysis process of **Athletix**, an academic project on athlete performance and training management. Your professional insights as a coach/trainer will help define training and performance-related system requirements.

All information will be used strictly for academic purposes.

Yours sincerely,

Sunandan Das – 23101218

Rokonur Zaman Rokon – 23101194

Md. Sakif Hossain – 23101219

Md Minhazul Islam – 23101207

Akamsur
24.02.26

Signature

Date: 24/02/26

To

Dr. Jahon Ara Akter

The Concerned Medical Officer

University of Asia Pacific

Respected Sir,

Subject: Request for Requirement Analysis Support of Athletix Project

I respectfully request your support in the requirement analysis process of **Athletix**, an academic athlete management project. Your medical expertise will help identify health, injury management, and recovery-related system requirements.

All information will be used strictly for academic purposes.

Yours sincerely,

Sunandan Das – 23101218

Rokonur Zaman Rokon – 23101194

Md. Sakif Hossain – 23101219

Md Minhazul Islam – 23101207

Dr. Jahon Ara Akter
24/02/26
Signature

Dr. Jahon Ara Akter
Medical Center
University of Asia Pacific
House 47A/A, Green Road
Dhaka-1205

Request for Professional Input on University Project.

2 messages

Rokonur Zaman Rokon <rrokonurzaman@gmail.com>
To: drtofael.dmck61@gmail.com <drtofael.dmck61@gmail.com>

Tue, Feb 24, 2026 at 13:30

Assalamu Alaikum, Bhaia.

I am currently working on a university project related to athlete health and performance monitoring. I would be grateful if you could kindly help me by answering the following questions. Your professional insights will greatly support my research.

1. What signs usually indicate that an athlete is at risk of injury?
2. What causes most sports injuries among student athletes?
3. How effectively are fatigue and recovery monitored in your training environment?
4. How would early detection of fatigue or pain improve performance and safety?
5. What features would you expect in a performance and health monitoring system?
6. How should a system notify risk of overtraining or injury?
7. How important is direct communication with medical staff through a digital platform?
8. What concerns would you have about using such a system?
9. How important is data privacy when sharing health and performance data?
10. How do athletes usually report pain or discomfort?
11. How quickly is medical attention provided after reporting an issue?
12. What challenges exist in managing training, recovery, and health?

Thank you for your time and assistance.

Kind regards,
Rokonur Zaman Rokon

Tofael Ahmed <drtofael.dmck61@gmail.com>
To: Rokonur Zaman Rokon <rrokonurzaman@gmail.com>

Tue, Feb 24, 2026 at 13:31

Dear Rokon,

Thank you for your email. Please find my responses to your questions below:

1. **What signs usually indicate that an athlete is at risk of injury?**
Common warning signs include persistent musculoskeletal pain, joint swelling, reduced range of motion, repeated muscle tightness, excessive fatigue, sleep disturbance, and decline in performance. These symptoms often precede major injuries.
2. **What causes most sports injuries among student athletes?**
The most common causes are overtraining, improper technique, inadequate warm-up, insufficient recovery time, poor conditioning, nutritional deficiencies, and ignoring early symptoms of pain or discomfort.
3. **How effectively are fatigue and recovery monitored in your training environment?**
In most student athletic settings, fatigue and recovery are monitored informally. Assessment is usually based on self-reporting rather than structured medical evaluation. Objective monitoring tools are often limited.
4. **How would early detection of fatigue or pain improve performance and safety?**
Early detection allows timely intervention, load adjustment, and appropriate treatment. This reduces the risk of severe injury, shortens recovery time, and maintains consistent athletic performance.
5. **What features would you expect in a performance and health monitoring system?**
A proper system should include daily symptom tracking, pain localization and severity scoring, training load monitoring, fatigue and recovery assessment, an alert mechanism for injury risk, secure medical data storage, and direct communication with healthcare providers.
6. **How should a system notify risk of overtraining or injury?**
The system should provide clear risk categorization (low, moderate, high), generate automated alerts, suggest corrective actions, and notify both the athlete and medical team for prompt evaluation.
7. **How important is direct communication with medical staff through a digital platform?**
It is highly important. Direct communication ensures early reporting, faster clinical assessment, continuous follow-up, and improved medical supervision.
8. **What concerns would you have about using such a system?**
Concerns may include data privacy, potential inaccuracies in automated assessments, overreliance on digital tools, and possible misinterpretation of health data by non-medical personnel.
9. **How important is data privacy when sharing health and performance data?**
Data privacy is critically important. Medical information must remain confidential and accessible only to authorized personnel with informed consent from the athlete.
10. **How do athletes usually report pain or discomfort?**
Athletes typically report symptoms verbally to coaches or medical staff. In some settings, digital reporting systems or written medical forms are used.
11. **How quickly is medical attention provided after reporting an issue?**
In organized institutions, evaluation usually occurs within 24 hours. In less structured environments, delays may occur due to limited medical resources.
12. **What challenges exist in managing training, recovery, and health?**
Major challenges include balancing academic workload with training, insufficient recovery time, delayed symptom reporting, limited access to sports medicine specialists, and pressure to continue participation despite discomfort.

I hope these responses will be helpful for your university project. Please feel free to contact me if you require any further clarification. Best of luck

Best regards,
Dr. Tofael
[Quoted text hidden]



